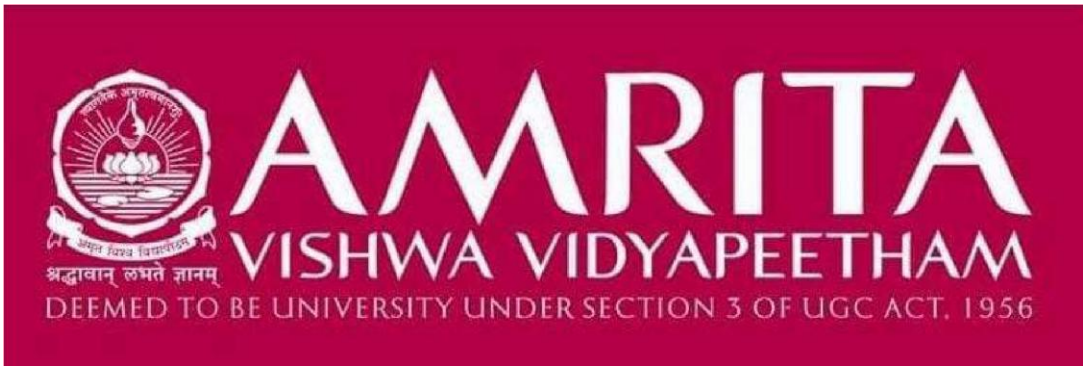
.




# DATA STRUCTURES AND ALGORITHMS 23CSE203 WEEK MANUAL

| | |
|---|---|
| **SUBMITTED BY** | |
| **NAME** | B.Karthikeya |
| **ROLL NO.** | Av.sc.u4cse25016 |
| **YEEAR/SEM/SECTION** | 2nd year /sem-3/Cse-A |
| **CREDITS** | 05 |

| | |
|---|---|
| **SUBMITTED TO** | DR.RAJ KUMAR BATCHU |
| **DESIGNATION** | ASST.PROFFESSOR(S l .Grade) |

.

INDEX

| | Title | Date | Remarks |
|---|---|---|---|
| | WEEK-1<br>1a). Rocket launch systems display the countdown of before launch instead of using a loop to implement the countdown using the recursion. Write a recursion Python program to display the countdown from n to 1 followed by launch.<br>1b).Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years.<br>1c).An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.<br>1d).In a logistics company, the number of possible ways to arrange different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number.<br>1e).Write a recursive Python program to print the first n terms of a Fibonacci series. | 23-07-2026 | |

.

| | | | |
|---|---|---|---|
| | WEEK-2<br>2a). Write a Python program to perform linear search om the given input.<br>2b.1). Write a python program to perform binary search on the given input verify by taking sorted array as a input.<br>2b.2). Write a python program to perform binary search on the given input to verify by taking Unsorted array as a input. | 30-07-2026 | |
| | WEEK-3<br>3a). Write a python program to sort the elements using a bubble sort technique. Take input elements from the user.<br>3b). Write a python program to sort the elements using an insertion sort technique. Take input elements from the user.<br>3c). Write a python program to sort the elements using a selection of sort technique. Take input elements from the user.<br>3d). Write a python program to sort the elements using a merge sort technique. Take input elements from the user.<br>3e).Write a python program to sort the elements using a quick sort of technique. Take input elements from the user. | 06-08-2026 | |

.

| | | | |
|---|---|---|---|
| | WEEK-4<br>4a).Write a Python program to implement a Singly Linked List with the following operations:<br>1.Create a singly linked list with n nodes.<br>2. Insert a node at the beginning of the linked list.<br>3. Insert a node at the end of the linked list.<br>4. Insert a node at a specific position.<br>5. Delete a node by its value.<br>6. Delete the first node.<br>7. Delete the last node.<br>8. Count the number of nodes in the linked list.<br>9. Display/Traverse the linked list.<br>10. Exit the program.<br>4b).To write a Python program to implement a Doubly Linked List and perform insertion, deletion, counting, and traversal operations in both forward and backward directions.<br>4c).To write a Python program to implement a Circular Linked List and perform insertion, deletion, counting, and traversal operations. | 13-08-2026 | |
| | WEEK-5<br>5a).Write a Python Program Implement a Stack using Array also Implement the Stack Operations Such as Push, | 27-08-2026 | |

.

| | | | | |
|---|---|---|---|---|
| | | Pop,Peak also display the Stack Elements.<br>5b).Write a Python Program Implement a Stack using Linked List also Implement the Stack Operations Such as Push, Pop,Peak also display the Stack Elements. | | |
| | | WEEK-6<br>6a).Write a Python Program to implement Queue and its operations such as In queue, Dequeue, Peek, Display with a Menu-driven approach implement using Array and Linked List data structure.<br>6b).Write a Python Program to implement Queue and its operations such as In queue, Dequeue, Peek, Display with a Menu-driven approach implement using Circular Queue. | 03-09-2026 | |
| | | WEEK-7<br>7a.1).Write a Python Program to implement a binary tree using an array with all of its operations.<br>7a.2).Write a Python Program to implement a binary tree using an linked list with all of its operations. | 24-09-2026 | |
| | | WEEK-8<br>8a).Write a Python Program to implement<br>Binary Search Tree with it's operations Transversals also | 08-10-2026 | |

.

| | | | | |
|---|---|---|---|---|
| | | WEEK-9<br>9a).Write a Python Program to implement<br>AVL Tree with it's operations<br>Transversals also. | 22-10-2026 | |
| | | WEEK-10<br>10a).Write a python program to implement hashing using seperate chaining linear and Quardtic Probic. | 29-10-2026 | |

.

## WEEK-1

**1A)AIM:-** rocket launch systems display the countdown of before launch instead of using a loop implement the countdown using the recursion write a recursion Python program to display the countdown from n to 1 followed by launch.

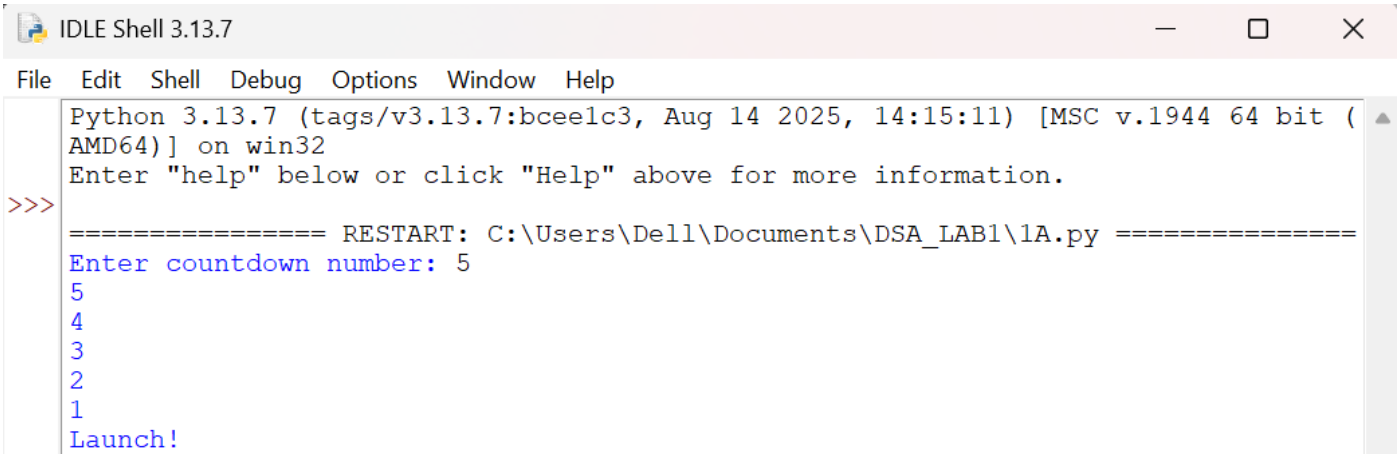
## PROGRAM:-

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)

# Input
n = int(input("Enter countdown number: "))

# Function call
countdown(n)
```

## OUTPUT:-



.

**EXPLANATION:-**

- def defines the countdown() function.
- input() accepts the countdown value from the user.
- int() converts the input into an integer.
- countdown() starts the recursive countdown process.
- if checks the base case (n == 0).
- else executes when the condition is False.
- print(n) displays the current countdown number.
- countdown(n - 1) recursively calls the function with the value decreased by 1.
- print("Launch!") displays the launch message when the countdown reaches 0.
- The program demonstrates the use of functions, recursion, conditional statements, input, and output to perform the countdown.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **2** | Missing base case (if n == 0) causing infinite recursion. | Include the base case to stop recursion .l |
| **3** | incorrect indentation leading to an Indentation Error | Maintain proper Python indentation. |
| **4** | Missing colon (:) after def, if, or else. | Add the required colon (:) after statements. |
| **5** | Calling countdown(n) instead of countdown(n - 1), causing infinite recursion. | Decrease the value by 1 in each recursive call using countdown(n - 1). |

**TIME COMPLEXITY:-** O(n)

**SPACE COMPLEXITY:-** O(n)

**REASON:-** Function is called once for each value from n to 0.

.

**1B)AIM:-** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P = Principal growth factor, n = number of years.

## PROGRAM:-

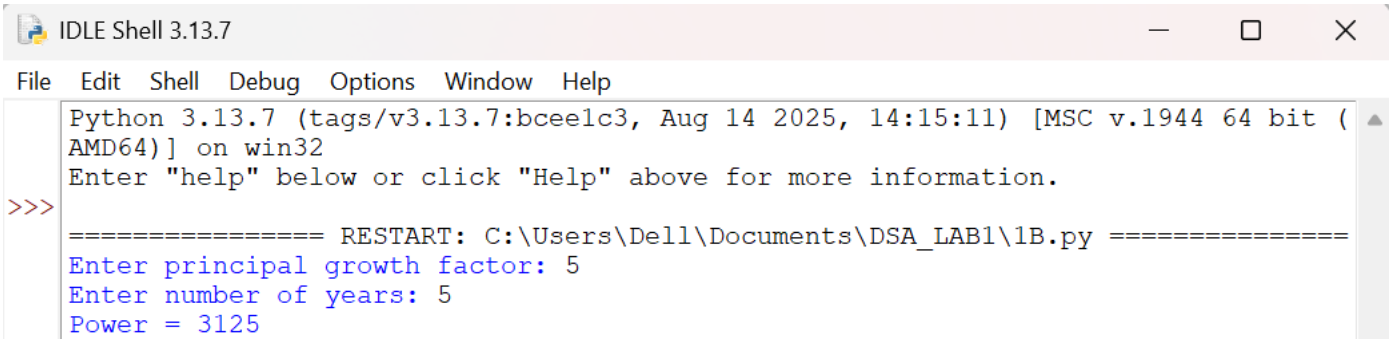
```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

# Input
p = int(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

# Function call
result = power(p, n)

# Output
print("Power =", result)
```

## OUTPUT:-



.

**EXPLANATION:-**

- power() is the recursive function that computes P raised to n.
- The base case is n == 0, which returns 1 (any number to the power 0 is 1).
- The recursive case multiplies p with power(p, n - 1), reducing n by 1 on each call.
- float() allows the principal growth factor to be a decimal value.
- input() collects the values of P and n from the user.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.
- print() displays the final computed value of P^n.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Missing base case (n == 0) causing infinite recursion. | Include the base case to stop recursion. |
| **2** | Entering a negative value for n causes the recursion to never reach 0. | Restrict input to non-negative integers or handle negative n separately. |
| **3** | Using int() for P truncates decimal growth factors. | Use float() to accept decimal values for P. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Calling power(p, n) instead of power(p, n - 1), causing infinite recursion. | Decrease n by 1 in each recursive call using power(p, n - 1). |

**TIME COMPLEXITY:-** O(n)

**SPACE COMPLEXITY:-** O(n)

**REASON:-** Recursive call is made n times to compute $P^n$.

.

**1C)AIM:-** An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

## PROGRAM:-

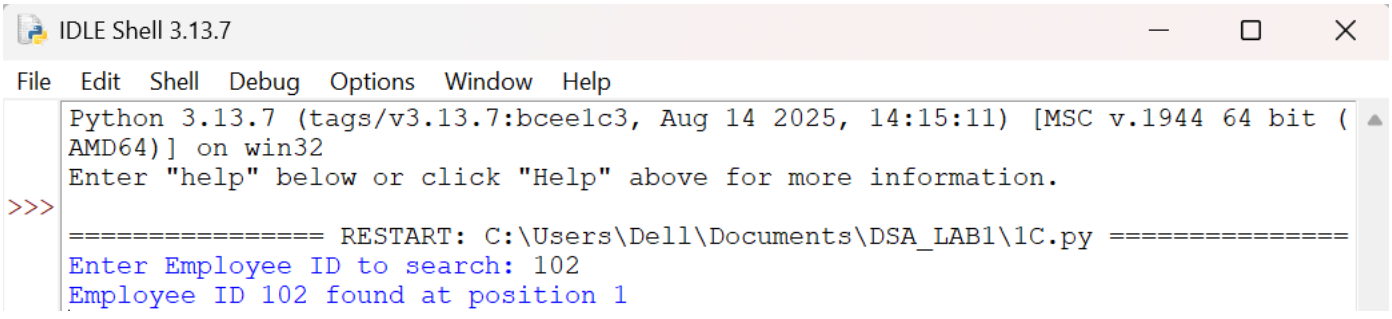
```
def search_employee(emp_list, emp_id, index=0):
    if index == len(emp_list):
        return -1
    elif emp_list[index] == emp_id:
        return index
    else:
        return search_employee(emp_list, emp_id, index + 1)


# Input
employee_ids = [101, 102, 103, 104, 105]
search_id = int(input("Enter Employee ID to search: "))

# Function call
position = search_employee(employee_ids, search_id)

if position != -1:
    print("Employee ID", search_id, "found at position", position)
else:
    print("Employee ID", search_id, "not found")
```

## OUTPUT:-



.

**EXPLANATION:-**

- search_employee() is the recursive function that searches the list one index at a time.
- The base case index == len(emp_list) means every element was checked and the ID was not found, so it returns -1.
- The second condition checks if the current element matches the employee ID being searched.
- If not matched, the function calls itself with index + 1, moving to the next employee in the list.
- employee_ids is the list of employee IDs stored by the HR department.
- input() and int() collect the employee ID to search for from the user.
- The if-else block reports whether the employee ID was found and at which position.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Missing base case (index == len(emp_list)) causing infinite recursion. | Include the base case to stop recursion when the list is exhausted. |
| **2** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **3** | Calling search_employee(emp_list, emp_id, index) without incrementing index, causing infinite recursion. | Increase index by 1 in each recursive call using index + 1. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Searching an empty list without handling it, giving a confusing result. | Add a check for an empty list before searching. |

**TIME COMPLEXITY:-** O(n)

**SPACE COMPLEXITY:-** O(n)

**REASON:-** In the worst case, every element in the list is checked recursively.

.

**1D)AIM:-** In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula. Develop a recursive program to calculate the factorial of a given number.

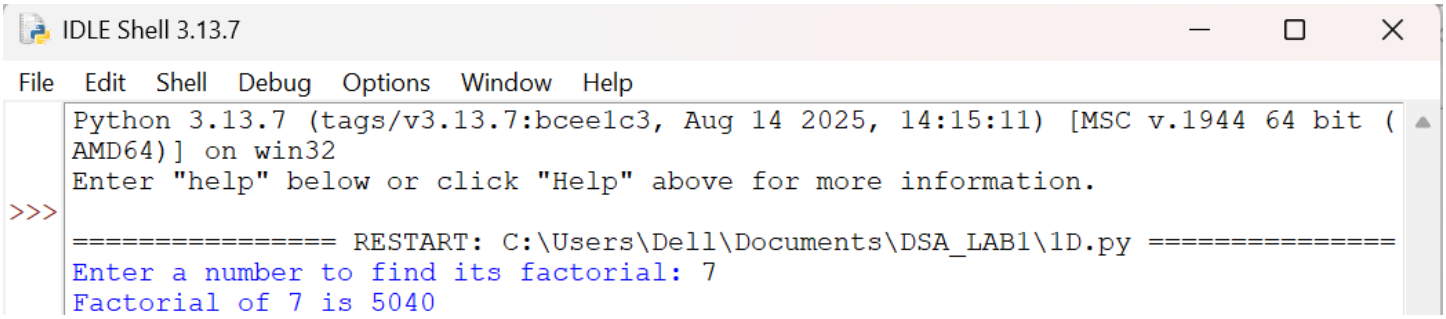
## PROGRAM:-

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)


# Input
n = int(input("Enter a number to find its factorial: "))

# Function call
print("Factorial of", n, "is", factorial(n))
```

## OUTPUT:-




## EXPLANATION:-

- factorial() is the recursive function that computes n! (n factorial).
- The base case is n == 0 or n == 1, both of which return 1.
- The recursive case multiplies n with factorial(n - 1), reducing n by 1 on each call.
- input() and int() collect the number whose factorial is to be found.
- Each recursive call is stacked until the base case is reached, then the results are multiplied back up the call stack.
- print() displays the number of possible arrangements (the factorial value).

.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Missing base case (n == 0 or n == 1) causing infinite recursion. | Include the base case to stop recursion. |
| **2** | Entering a negative number, for which factorial is undefined. | Restrict input to non-negative integers. |
| **3** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Calling factorial(n) instead of factorial(n - 1), causing infinite recursion. | Decrease n by 1 in each recursive call using factorial(n - 1). |

**TIME COMPLEXITY:-** O(n)

**SPACE COMPLEXITY:-** O(n)

**REASON:-** Function calls itself n times until reaching the base case.

.

**1E)AIM:-** Write a recursive Python program to print the first n terms of a Fibonacci series.

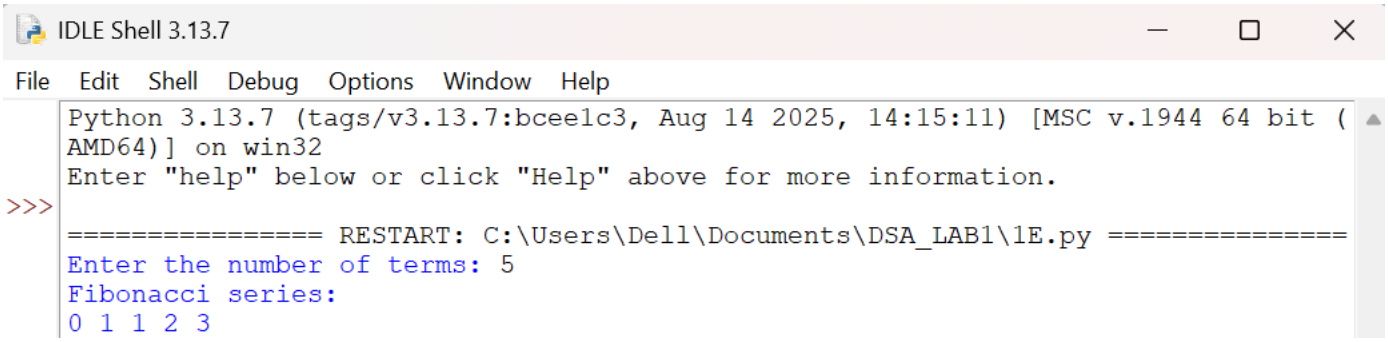
## PROGRAM:-

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)


# Input
n = int(input("Enter the number of terms: "))

# Function call
print("Fibonacci series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

## OUTPUT:-



.

**EXPLANATION:-**

- fibonacci() is the recursive function that returns the nth Fibonacci term.
- The base cases are n == 0 (returns 0) and n == 1 (returns 1).
- The recursive case adds fibonacci(n - 1) and fibonacci(n - 2), following the Fibonacci rule.
- input() and int() collect the number of terms to be printed.
- The for loop calls fibonacci(i) for each index from 0 to n - 1.
- print(..., end=" ") displays each term on the same line, separated by a space.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Missing base cases (n == 0 and n == 1) causing infinite recursion. | Include both base cases to stop recursion. |
| **2** | Entering a negative number of terms. | Restrict input to non-negative integers. |
| **3** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Using fibonacci(n - 1) - fibonacci(n - 2) instead of addition, giving wrong terms. | Use addition, fibonacci(n - 1) + fibonacci(n - 2), as per the Fibonacci rule. |

**TIME COMPLEXITY:-** $O(2^n)$

**SPACE COMPLEXITY:-** O(n)

**REASON:-** Each call creates two recursive calls, resulting in exponential time, while the recursion depth is n.

.

## WEEK-2

**2A)AIM:-** Write a Python program to perform linear search for the given input.

## PROGRAM:-

```
def linear_search(arr, target):
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1

# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

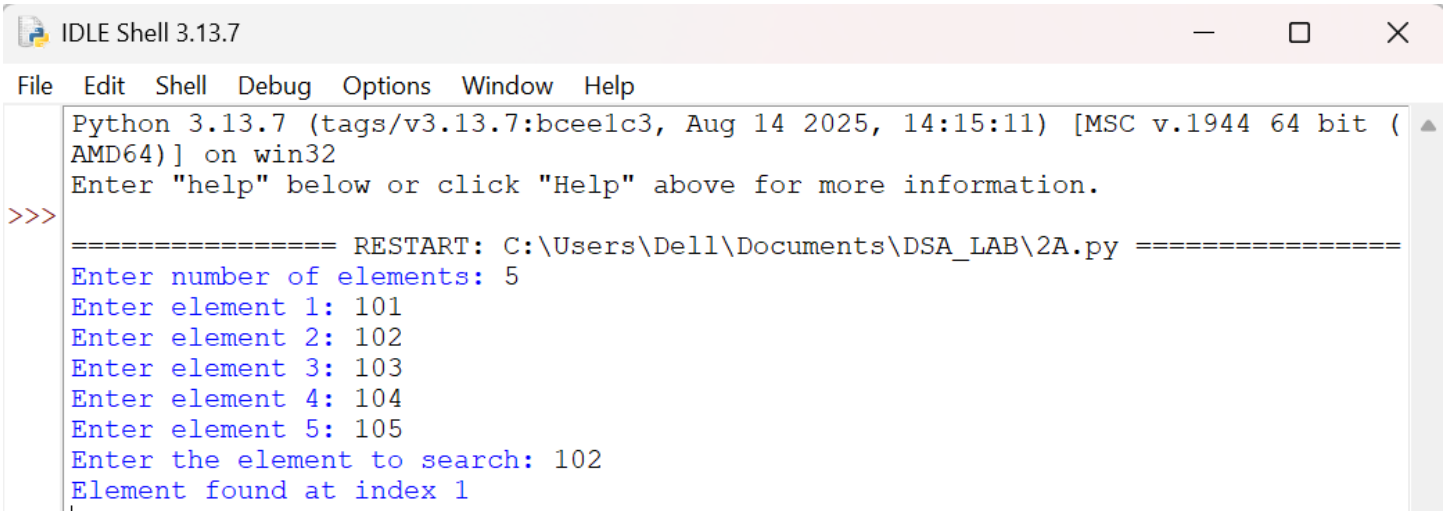
target = int(input("Enter the element to search: "))

# Function call
result = linear_search(arr, target)

# Output
if result != -1:
    print("Element found at index", result)
else:
    print("Element not found in the list")
```

## OUTPUT:-

.




**EXPLANATION:-**

- linear_search() checks each element of the list one by one from the beginning.
- The for loop iterates over every index of the array using range(len(arr)).
- If arr[index] equals the target, the matching index is returned immediately.
- If the loop finishes without a match, -1 is returned to indicate the element is not present.
- input() and int() collect the number of elements, the elements themselves, and the target value.
- The if-else block reports whether the target was found and at which index, or that it was not found.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **2** | Entering more or fewer elements than the specified count n. | Ensure exactly n elements are entered, one per prompt. |
| **3** | Searching an empty list (n = 0) without handling it. | Add a check for an empty list before searching. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |

.

| | | |
|---|---|---|
| **5** | Using == to compare elements of different types (e.g. int vs str), causing a false "not found". | Ensure the target and list elements are converted to the same type (int) before comparing. |

**TIME COMPLEXITY:-** O(n)

**SPACE COMPLEXITY:-** O(1)

**REASON:-** In the worst case, every element in the list is checked once; no extra space is used besides a few variables.

.

**2B)AIM:-** Write a Python program to perform binary search of the given input, verifying by taking a sorted array and an unsorted array as input.

## PROGRAM:-

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

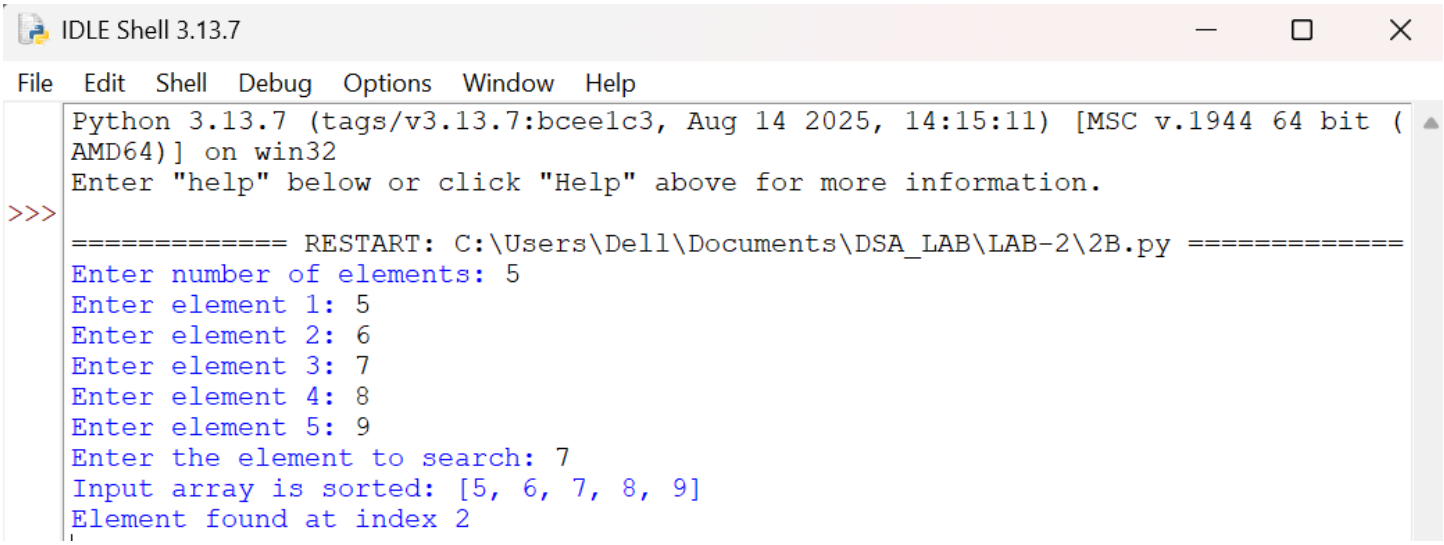
# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

target = int(input("Enter the element to search: "))

# Verify whether the input array is sorted or unsorted
if arr == sorted(arr):
    print("Input array is sorted:", arr)
    result = binary_search(arr, target)
    if result != -1:
        print("Element found at index", result)
    else:
        print("Element not found in the list")
else:
    print("Input array is NOT sorted:", arr)
    print("Binary search requires a sorted array.")
    print("Result on unsorted array cannot be trusted. Sorting the array first...")
    sorted_arr = sorted(arr)
    print("Sorted array:", sorted_arr)
    result = binary_search(sorted_arr, target)
    if result != -1:
```

```
    print("Element found at index", result, "in the sorted array")
else:
    print("Element not found in the list")
```

## OUTPUT:-




## EXPLANATION:-

- binary_search() repeatedly narrows the search range between low and high using the midpoint mid.
- If arr[mid] equals the target, the index mid is returned immediately.
- If arr[mid] is less than the target, the search continues in the right half by moving low to mid + 1.
- Otherwise, the search continues in the left half by moving high to mid - 1.
- arr == sorted(arr) checks whether the entered array is already sorted, to verify binary search's precondition.
- If the array is sorted, binary search runs directly on it and reports the result.
- If the array is unsorted, the program shows that binary search cannot be trusted on it, sorts the array with sorted(), and re-runs binary search on the sorted copy.

## ERRORS:-

| SNO | Errors Observed | Rectification |
|---|---|---|

.

| | | |
|---|---|---|
| **1** | Running binary search directly on an unsorted array gives an unreliable or wrong result. | Always verify with arr == sorted(arr) and sort the array before searching. |
| **2** | Using mid = (low + high) / 2 gives a float, causing a TypeError when indexing the list. | Use integer floor division (//) instead of / to compute mid. |
| **3** | Using low < high instead of low <= high, missing the case where low equals high. | Use low <= high so a single remaining element is still checked. |
| **4** | Not updating low or high inside the loop, causing an infinite loop. | Always set low = mid + 1 or high = mid - 1 inside the loop body. |
| **5** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |

**TIME COMPLEXITY:-** O(log n)

**SPACE COMPLEXITY:-** O(1)

**REASON:-** The search range is halved on every iteration, so the number of comparisons grows logarithmically with n; only a few variables (low, high, mid) are used regardless of input size.

.

## WEEK-3

**3A)AIM:-** Write a Python program to bubble sort the elements, taking input from the user.

## PROGRAM:-

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

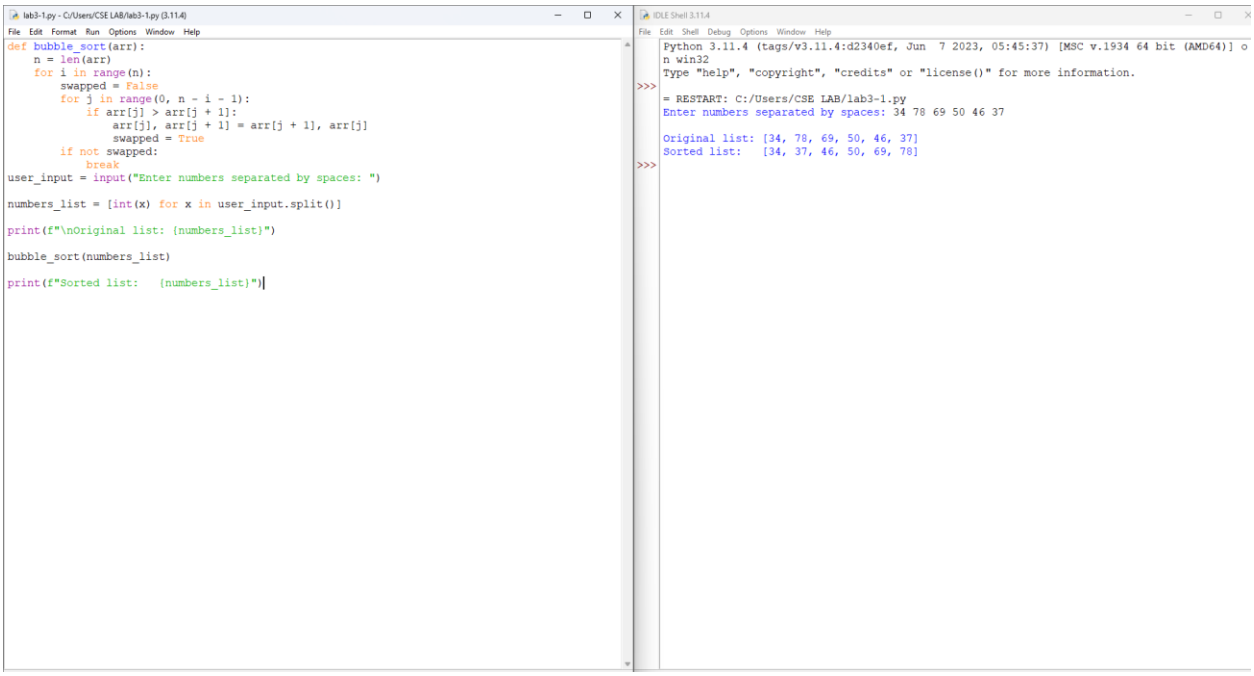

# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = bubble_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

## OUTPUT:-

•



```
lab3-1.py - C:/Users/CSE LAB/lab3-1.py (3.11.4)
File  Edit  Format  Run  Options  Window  Help
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
user_input = input("Enter numbers separated by spaces: ")

numbers_list = [int(x) for x in user_input.split()]

print(f"\nOriginal list: {numbers_list}")

bubble_sort(numbers_list)

print(f"Sorted list:   {numbers_list}")
```

```
IDLE Shell 3.11.4
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] o
n win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/CSE LAB/lab3-1.py
Enter numbers separated by spaces: 34 78 69 50 46 37

Original list: [34, 78, 69, 50, 46, 37]
Sorted list:   [34, 37, 46, 50, 69, 78]
>>>
```

## EXPLANATION:-

- bubble_sort() repeatedly steps through the list, comparing adjacent elements.
- The outer loop runs n - 1 times, controlling the number of passes needed.
- The inner loop compares arr[j] and arr[j + 1], swapping them if they are out of order.
- After each pass, the largest remaining unsorted element "bubbles up" to its correct position at the end.
- input() and int() collect the number of elements and each element from the user.
- print() displays the final sorted array.

## ERRORS:-

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **2** | Entering more or fewer elements than the specified count n. | Ensure exactly n elements are entered, one per prompt. |

.

| | | |
|---|---|---|
| **3** | Using range(n) instead of range(n - 1 - i) in the inner loop, causing an IndexError. | Use range(n - 1 - i) so arr[j + 1] never goes out of bounds. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Swapping elements incorrectly without simultaneous assignment, losing a value. | Use arr[j], arr[j + 1] = arr[j + 1], arr[j] for a correct swap. |

**TIME COMPLEXITY:-** $O(n^2)$

**SPACE COMPLEXITY:-** $O(1)$

**REASON:-** Two nested loops compare and swap every pair of elements in the worst case; sorting is done in place using only a few extra variables.

.

**3B)AIM:-** Write a Python program to insertion sort the elements, taking input from the user.

## PROGRAM:-

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

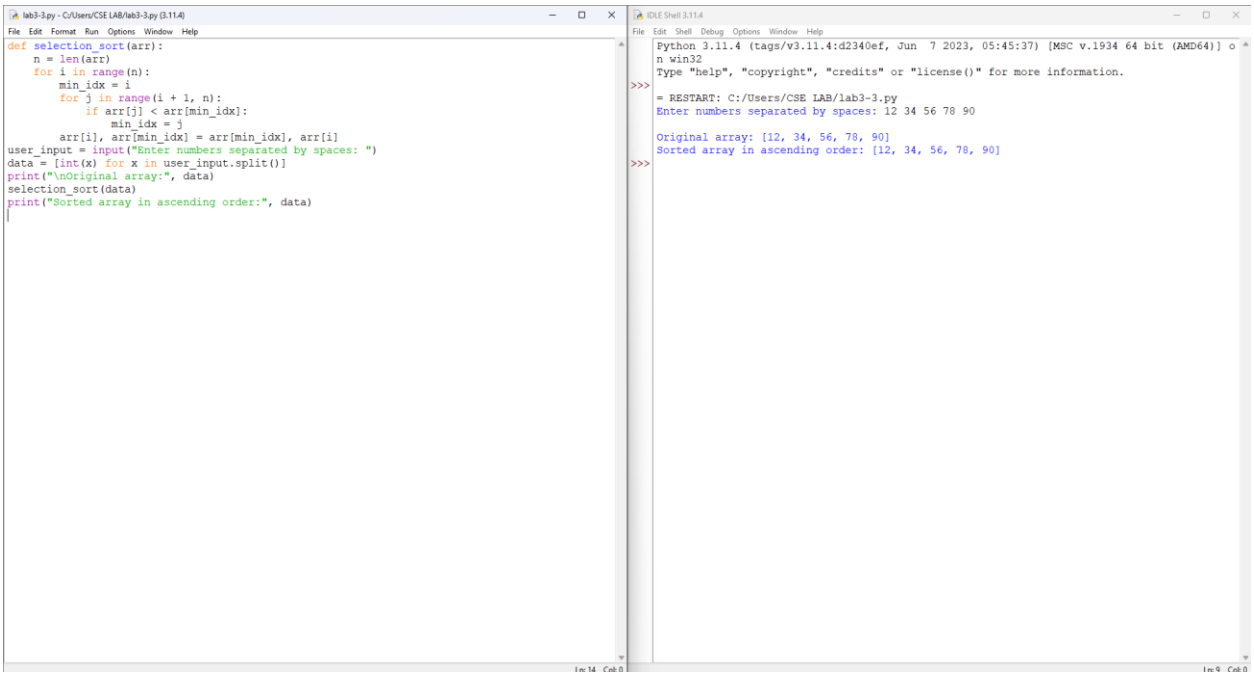

# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = insertion_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

## OUTPUT:-

•




**EXPLANATION:-**

- insertion_sort() builds the sorted list one element at a time, similar to sorting playing cards in hand.
- key stores the current element that is being inserted into the sorted portion of the array.
- The while loop shifts elements greater than key one position to the right.
- j decreases until the correct position for key is found or the start of the array is reached.
- arr[j + 1] = key places the element in its correct sorted position.
- input() and int() collect the number of elements and each element from the user.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **2** | Entering more or fewer elements than the specified count n. | Ensure exactly n elements are entered, one per prompt. |

.

| | | |
|---|---|---|
| **3** | Missing the boundary check j >= 0, causing an IndexError. | Always check j >= 0 before comparing arr[j] inside the while loop. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Forgetting to decrement j inside the while loop, causing an infinite loop. | Decrease j by 1 in each iteration using j -= 1. |

**TIME COMPLEXITY:-** $O(n^2)$

**SPACE COMPLEXITY:-** $O(1)$

**REASON:-** In the worst case (reverse sorted input), every element is compared against and shifted through the entire sorted portion; sorting is done in place.

.

**3C)AIM:-** Write a Python program to selection sort the elements, taking input from the user.

## PROGRAM:-

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr

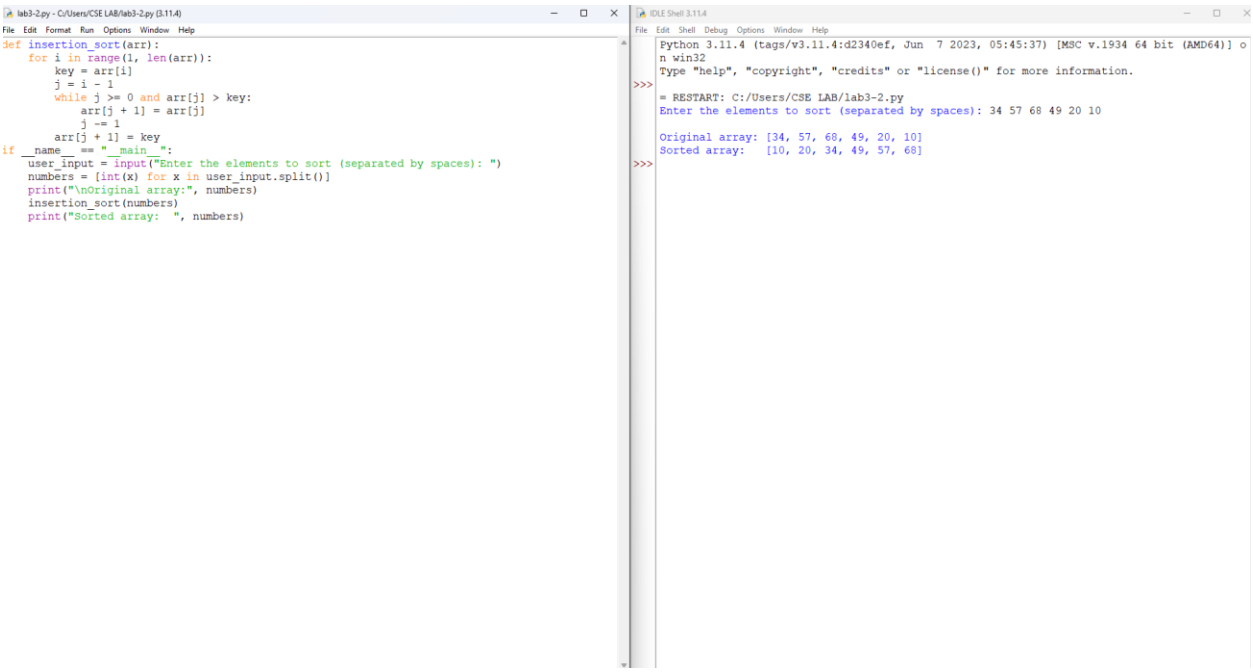

# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = selection_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

## OUTPUT:-

•



```
lab3-2.py - C:/Users/CSE LAB/lab3-2.py (3.11.4)
File  Edit  Format  Run  Options  Window  Help
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
if __name__ == "__main__":
    user_input = input("Enter the elements to sort (separated by spaces): ")
    numbers = [int(x) for x in user_input.split()]
    print("\nOriginal array:", numbers)
    insertion_sort(numbers)
    print("Sorted array:  ", numbers)
```

```
IDLE Shell 3.11.4
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] o
n win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/CSE LAB/lab3-2.py
Enter the elements to sort (separated by spaces): 34 57 68 49 20 10

Original array: [34, 57, 68, 49, 20, 10]
Sorted array:   [10, 20, 34, 49, 57, 68]
>>>
```

**EXPLANATION:-**

- selection_sort() repeatedly selects the minimum element from the unsorted part and places it at the beginning.
- min_index tracks the index of the smallest element found during the current pass.
- The inner loop scans the unsorted portion of the array to find the position of the minimum value.
- arr[i], arr[min_index] = arr[min_index], arr[i] swaps the found minimum into its correct sorted position.
- input() and int() collect the number of elements and each element from the user.
- print() displays the final sorted array.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |

.

| | | |
|---|---|---|
| **2** | Entering more or fewer elements than the specified count n. | Ensure exactly n elements are entered, one per prompt. |
| **3** | Not resetting min_index to i at the start of each outer pass, giving a wrong sort. | Reset min_index = i at the beginning of every outer loop iteration. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Using <= instead of < when comparing arr[j] and arr[min_index], causing unnecessary swaps. | Use strict < so equal elements are not swapped needlessly. |

**TIME COMPLEXITY:-** $O(n^2)$

**SPACE COMPLEXITY:-** $O(1)$

**REASON:-** The inner loop scans all remaining unsorted elements for every outer iteration, giving $n^2$ comparisons; sorting is done in place.

.

**3D)AIM:-** Write a Python program to merge sort the elements, taking input from the user.

## PROGRAM:-

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

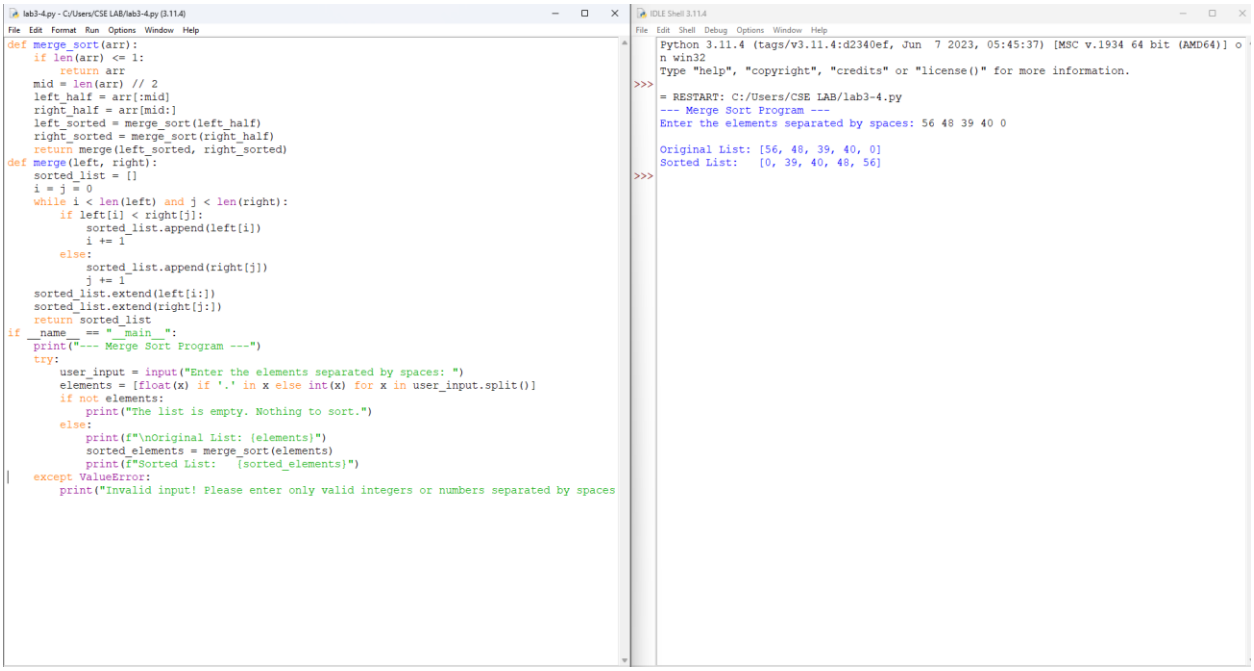

# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = merge_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

.

## OUTPUT:-



```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]
    left_sorted = merge_sort(left_half)
    right_sorted = merge_sort(right_half)
    return merge(left_sorted, right_sorted)
def merge(left, right):
    sorted_list = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1
    sorted_list.extend(left[i:])
    sorted_list.extend(right[j:])
    return sorted_list
if __name__ == "__main__":
    print("--- Merge Sort Program ---")
    try:
        user_input = input("Enter the elements separated by spaces: ")
        elements = [float(x) if '.' in x else int(x) for x in user_input.split()]
        if not elements:
            print("The list is empty. Nothing to sort.")
        else:
            print(f"\nOriginal List: {elements}")
            sorted_elements = merge_sort(elements)
            print(f"Sorted List:   {sorted_elements}")
    except ValueError:
        print("Invalid input! Please enter only valid integers or numbers separated by spaces
```

```
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] o
n win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/CSE LAB/lab3-4.py
--- Merge Sort Program ---
Enter the elements separated by spaces: 56 48 39 40 0

Original List: [56, 48, 39, 40, 0]
Sorted List:   [0, 39, 40, 48, 56]
>>>
```

## EXPLANATION:-

- merge_sort() recursively splits the array into halves until each sub-array has one element.
- mid = len(arr) // 2 finds the midpoint used to divide the array into left and right halves.
- left and right hold the recursively sorted halves of the array.
- merge() combines two sorted halves into a single sorted list by comparing their front elements.
- result.extend(...) appends any elements left over once one half is fully consumed.
- input() and int() collect the number of elements and each element from the user.

## ERRORS:-

| SNO | Errors Observed | Rectification |
|---|---|---|
| 1 | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |

.

| | | |
|---|---|---|
| **2** | Entering more or fewer elements than the specified count n. | Ensure exactly n elements are entered, one per prompt. |
| **3** | Missing base case (len(arr) <= 1), causing infinite recursion. | Include the base case to stop recursion when the sub-array has one or zero elements. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Forgetting to extend the remaining elements after the while loop in merge(), losing elements. | Use result.extend(left[i:]) and result.extend(right[j:]) after the loop. |

**TIME COMPLEXITY:-** O(n log n)

**SPACE COMPLEXITY:-** O(n)

**REASON:-** The array is divided log n times and each level requires O(n) work to merge the halves; additional arrays are created at every recursive call.

.

**3E)AIM:-** Write a Python program to quick sort the elements, taking input from the user.

## PROGRAM:-

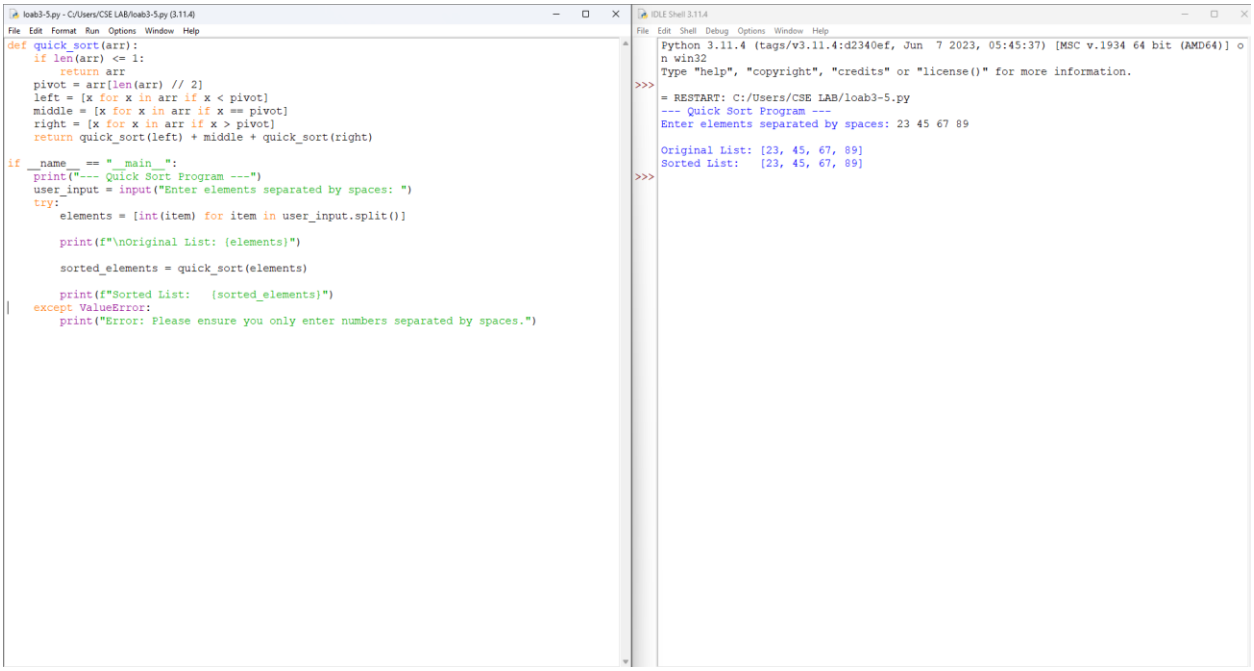
```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)


# Input
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))


# Function call
sorted_arr = quick_sort(arr)


# Output
print("Sorted array:", sorted_arr)
```

## OUTPUT:-



```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

if __name__ == "__main__":
    print("--- Quick Sort Program ---")
    user_input = input("Enter elements separated by spaces: ")
    try:
        elements = [int(item) for item in user_input.split()]

        print(f"\nOriginal List: {elements}")

        sorted_elements = quick_sort(elements)

        print(f"Sorted List:   {sorted_elements}")
    except ValueError:
        print("Error: Please ensure you only enter numbers separated by spaces.")
```

```
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] o
n win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/CSE LAB/loab3-5.py
--- Quick Sort Program ---
Enter elements separated by spaces: 23 45 67 89

Original List: [23, 45, 67, 89]
Sorted List:   [23, 45, 67, 89]
>>>
```

**EXPLANATION:-**

- quick_sort() recursively partitions the array around a chosen pivot element.
- pivot = arr[len(arr) // 2] selects the middle element of the array as the pivot for partitioning.
- left, middle, and right use list comprehensions to group elements less than, equal to, and greater than the pivot.
- The function recursively sorts left and right and concatenates them with middle in between.
- input() and int() collect the number of elements and each element from the user.
- print() displays the final sorted array.

**ERRORS:-**

| SNO | Errors Observed | Rectification |
|---|---|---|
| **1** | Invalid (non-numeric) input entered by the user. | Enter a valid integer as input. |
| **2** | Entering more or fewer elements than the specified count n. | Ensure exactly n elements are entered, one per prompt. |

.

| | | |
|---|---|---|
| **3** | Missing base case (len(arr) <= 1), causing infinite recursion. | Include the base case to stop recursion when the sub-array has one or zero elements. |
| **4** | Incorrect indentation leading to an IndentationError. | Maintain proper Python indentation. |
| **5** | Always choosing the first element as pivot on already-sorted input, degrading performance to $O(n^2)$. | Choose a better pivot strategy, such as the middle element or a random element. |

**TIME COMPLEXITY:-** $O(n \log n)$ average, $O(n^2)$ worst case

**SPACE COMPLEXITY:-** $O(\log n)$

**REASON:-** On average the pivot splits the array into two roughly equal halves, giving log n levels of recursion with O(n) work per level; the recursion stack depth is O(log n) on average.

.

## WEEK-4

**AIM:-** Write a Python program to implement a singly linked list that performs the following operations, taking input from the user: create a linked list, insert at the beginning, insert at the end, insert at a specific index, delete by value, delete the first node, delete the last node, count the number of nodes, display/traverse the list, and exit.

## PROGRAM:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def create_list(self):
        self.head = None
        n = int(input("Enter number of nodes to create: "))
        for i in range(n):
            data = int(input(f"Enter value for node {i + 1}: "))
            self.insert_at_end(data)
        print("Linked list created successfully.")

    def insert_at_beginning(self):
        data = int(input("Enter value to insert at beginning: "))
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print(f"{data} inserted at the beginning.")

    def insert_at_end(self, data=None):
        if data is None:
            data = int(input("Enter value to insert at end: "))
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
```

```
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node
        if data is not None:
            print(f"{data} inserted at the end.")

    def insert_at_index(self):
        index = int(input("Enter index to insert at (0-based): "))
        data = int(input("Enter value to insert: "))
        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            print(f"{data} inserted at index {index}.")
            return
        temp = self.head
        count = 0
        while temp is not None and count < index - 1:
            temp = temp.next
            count += 1
        if temp is None:
            print("Index out of range.")
            return
        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node
        print(f"{data} inserted at index {index}.")

    def delete_by_value(self):
        value = int(input("Enter value to delete: "))
        if self.head is None:
            print("List is empty.")
            return
        if self.head.data == value:
            self.head = self.head.next
            print(f"Value {value} deleted.")
            return
        prev = self.head
        curr = self.head.next
        while curr:
            if curr.data == value:
                prev.next = curr.next
```

```
            print(f"Value {value} deleted.")
            return
        prev = curr
        curr = curr.next
    print(f"Value {value} not found in list.")

def delete_first_node(self):
    if self.head is None:
        print("List is empty.")
        return
    deleted = self.head.data
    self.head = self.head.next
    print(f"Deleted first node with value {deleted}.")

def delete_last_node(self):
    if self.head is None:
        print("List is empty.")
        return
    if self.head.next is None:
        deleted = self.head.data
        self.head = None
        print(f"Deleted last node with value {deleted}.")
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
    deleted = temp.next.data
    temp.next = None
    print(f"Deleted last node with value {deleted}.")

def count_nodes(self):
    count = 0
    temp = self.head
    while temp:
        count += 1
        temp = temp.next
    print(f"Number of nodes: {count}")

def display(self):
    if self.head is None:
        print("List is empty.")
        return
    temp = self.head
    elements = []
```

```
        .

        while temp:
            elements.append(str(temp.data))
            temp = temp.next
        print(" -> ".join(elements) + " -> None")


# Main menu-driven program
sll = SinglyLinkedList()

while True:
    print("\n----- Singly Linked List Menu -----")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display/Traversal")
    print("10. Exit")

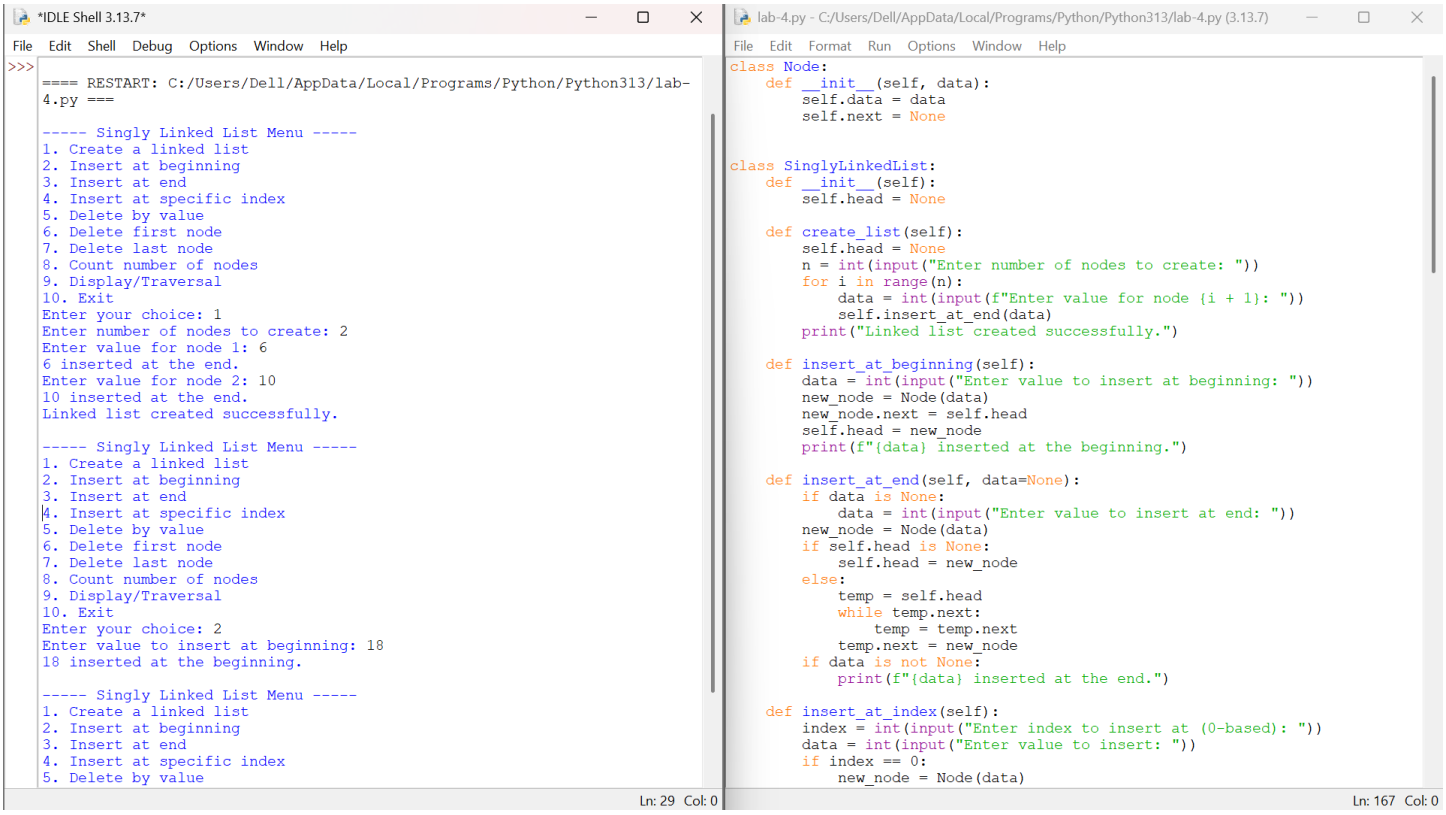
    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create_list()
    elif choice == 2:
        sll.insert_at_beginning()
    elif choice == 3:
        sll.insert_at_end()
    elif choice == 4:
        sll.insert_at_index()
    elif choice == 5:
        sll.delete_by_value()
    elif choice == 6:
        sll.delete_first_node()
    elif choice == 7:
        sll.delete_last_node()
    elif choice == 8:
        sll.count_nodes()
    elif choice == 9:
        sll.display()
    elif choice == 10:
        print("Exiting program.")
```

•

```
        break
    else:
        print("Invalid choice. Try again.")
```

# OUTPUT:-



```
*IDLE Shell 3.13.7*
File  Edit  Shell  Debug  Options  Window  Help
>>>
==== RESTART: C:/Users/Dell/AppData/Local/Programs/Python/Python313/lab-4.py ===

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 1
Enter number of nodes to create: 2
Enter value for node 1: 6
6 inserted at the end.
Enter value for node 2: 10
10 inserted at the end.
Linked list created successfully.

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 2
Enter value to insert at beginning: 18
18 inserted at the beginning.

----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
                                                    Ln: 29  Col: 0
```

```
lab-4.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/lab-4.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

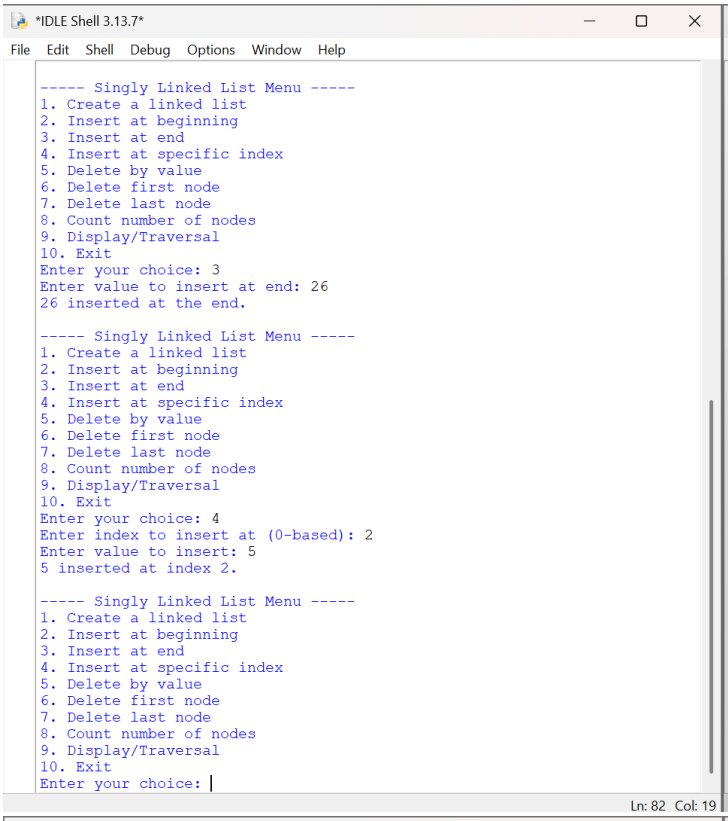
    def create_list(self):
        self.head = None
        n = int(input("Enter number of nodes to create: "))
        for i in range(n):
            data = int(input(f"Enter value for node {i + 1}: "))
            self.insert_at_end(data)
        print("Linked list created successfully.")

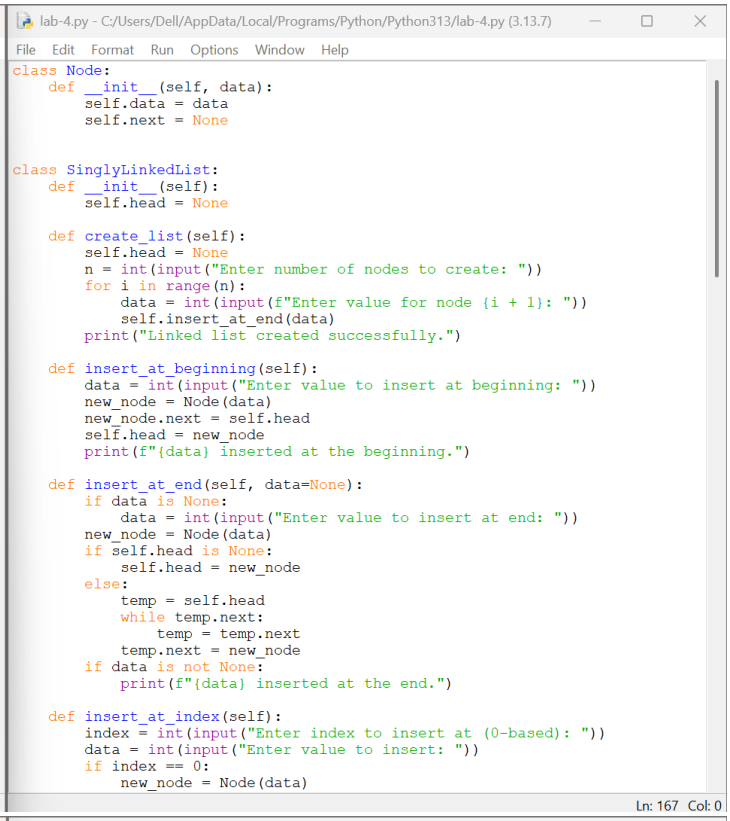
    def insert_at_beginning(self):
        data = int(input("Enter value to insert at beginning: "))
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print(f"{data} inserted at the beginning.")

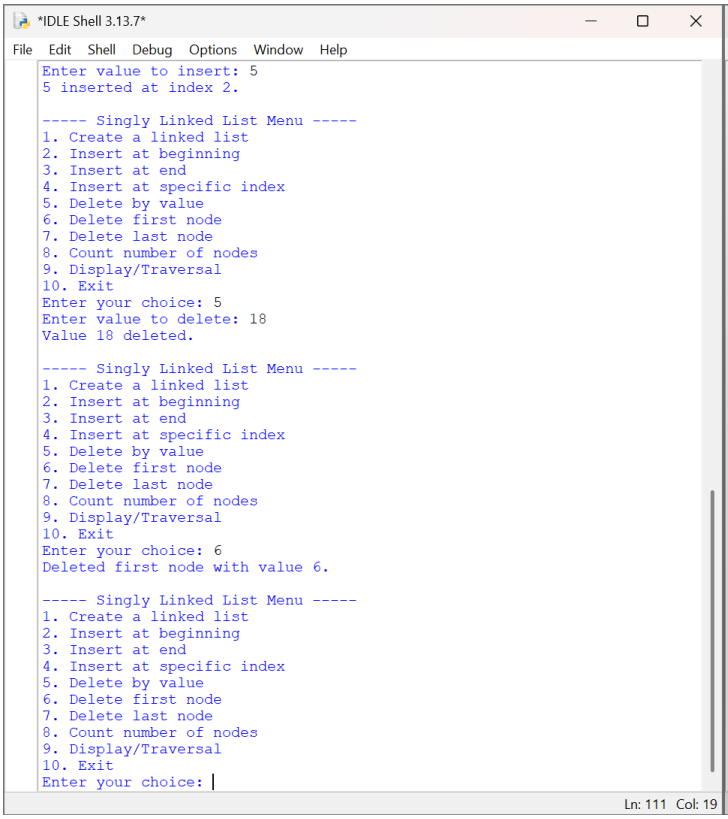
    def insert_at_end(self, data=None):
        if data is None:
            data = int(input("Enter value to insert at end: "))
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node
        if data is not None:
            print(f"{data} inserted at the end.")

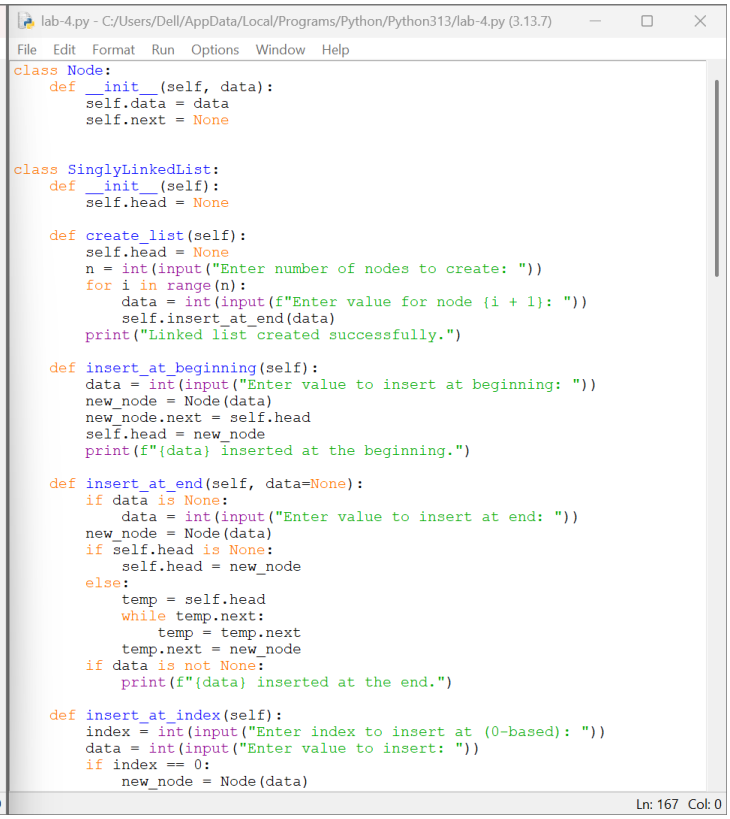
    def insert_at_index(self):
        index = int(input("Enter index to insert at (0-based): "))
        data = int(input("Enter value to insert: "))
        if index == 0:
            new_node = Node(data)
                                                    Ln: 167  Col: 0
```

•

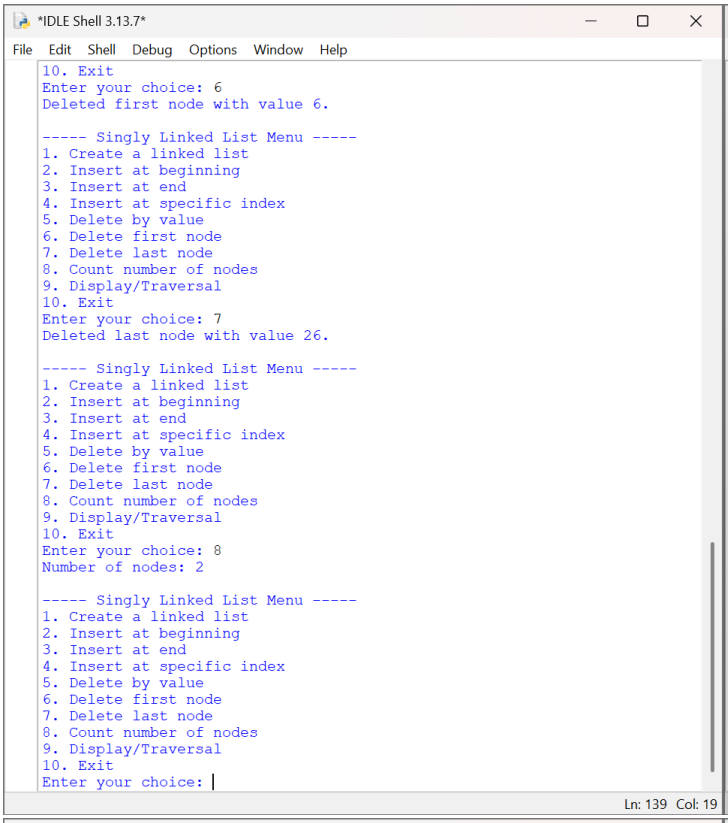

*IDLE Shell 3.13.7*
File Edit Shell Debug Options Window Help
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 3
Enter value to insert at end: 26
26 inserted at the end.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 4
Enter index to insert at (0-based): 2
Enter value to insert: 5
5 inserted at index 2.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice:
Ln: 82 Col: 19

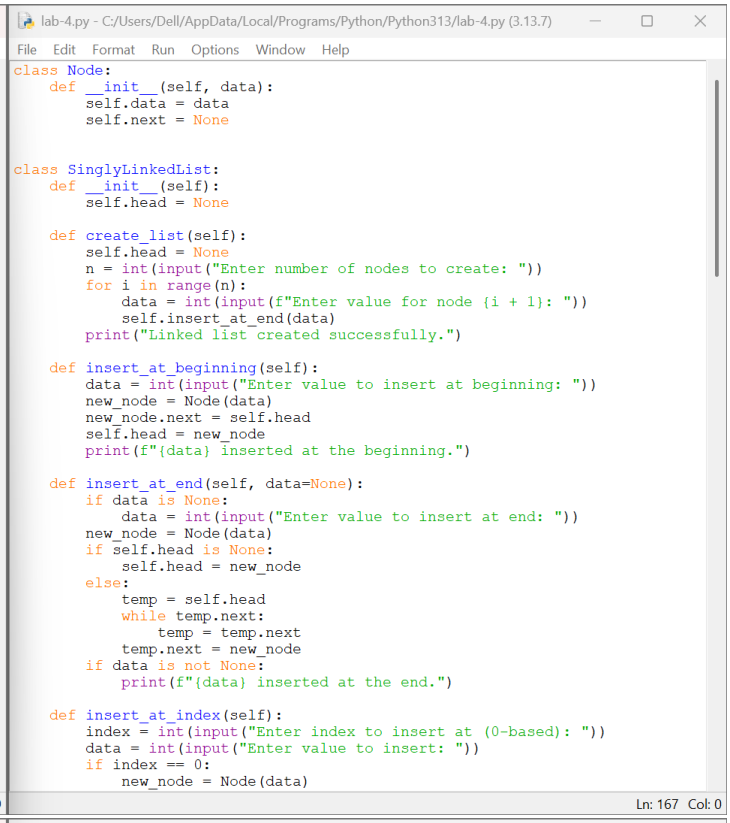

lab-4.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/lab-4.py (3.13.7)
File Edit Format Run Options Window Help
class Node:
def __init__(self, data):
self.data = data
self.next = None
class SinglyLinkedList:
def __init__(self):
self.head = None
def create_list(self):
self.head = None
n = int(input("Enter number of nodes to create: "))
for i in range(n):
data = int(input(f"Enter value for node {i + 1}: "))
self.insert_at_end(data)
print("Linked list created successfully.")
def insert_at_beginning(self):
data = int(input("Enter value to insert at beginning: "))
new_node = Node(data)
new_node.next = self.head
self.head = new_node
print(f"{data} inserted at the beginning.")
def insert_at_end(self, data=None):
if data is None:
data = int(input("Enter value to insert at end: "))
new_node = Node(data)
if self.head is None:
self.head = new_node
else:
temp = self.head
while temp.next:
temp = temp.next
temp.next = new_node
if data is not None:
print(f"{data} inserted at the end.")
def insert_at_index(self):
index = int(input("Enter index to insert at (0-based): "))
data = int(input("Enter value to insert: "))
if index == 0:
new_node = Node(data)
Ln: 167 Col: 0

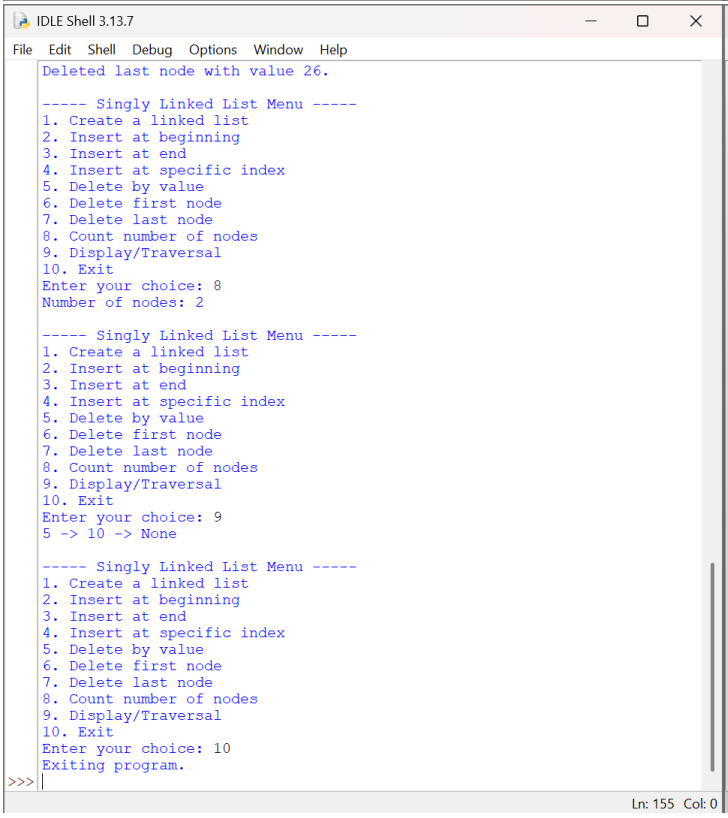

*IDLE Shell 3.13.7*
File Edit Shell Debug Options Window Help
Enter value to insert: 5
5 inserted at index 2.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 5
Enter value to delete: 18
Value 18 deleted.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 6
Deleted first node with value 6.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice:
Ln: 111 Col: 19

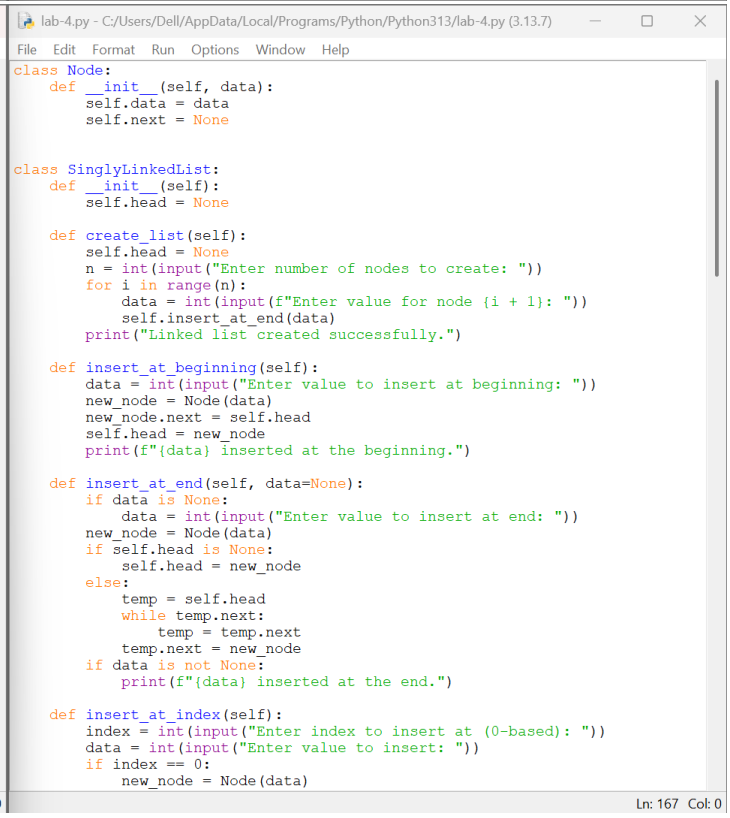

lab-4.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/lab-4.py (3.13.7)
File Edit Format Run Options Window Help
class Node:
def __init__(self, data):
self.data = data
self.next = None
class SinglyLinkedList:
def __init__(self):
self.head = None
def create_list(self):
self.head = None
n = int(input("Enter number of nodes to create: "))
for i in range(n):
data = int(input(f"Enter value for node {i + 1}: "))
self.insert_at_end(data)
print("Linked list created successfully.")
def insert_at_beginning(self):
data = int(input("Enter value to insert at beginning: "))
new_node = Node(data)
new_node.next = self.head
self.head = new_node
print(f"{data} inserted at the beginning.")
def insert_at_end(self, data=None):
if data is None:
data = int(input("Enter value to insert at end: "))
new_node = Node(data)
if self.head is None:
self.head = new_node
else:
temp = self.head
while temp.next:
temp = temp.next
temp.next = new_node
if data is not None:
print(f"{data} inserted at the end.")
def insert_at_index(self):
index = int(input("Enter index to insert at (0-based): "))
data = int(input("Enter value to insert: "))
if index == 0:
new_node = Node(data)
Ln: 167 Col: 0

•


*IDLE Shell 3.13.7*
File Edit Shell Debug Options Window Help
10. Exit
Enter your choice: 6
Deleted first node with value 6.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 7
Deleted last node with value 26.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 8
Number of nodes: 2
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice:
Ln: 139 Col: 19


lab-4.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/lab-4.py (3.13.7)
File Edit Format Run Options Window Help
class Node:
def __init__(self, data):
self.data = data
self.next = None
class SinglyLinkedList:
def __init__(self):
self.head = None
def create_list(self):
self.head = None
n = int(input("Enter number of nodes to create: "))
for i in range(n):
data = int(input(f"Enter value for node {i + 1}: "))
self.insert_at_end(data)
print("Linked list created successfully.")
def insert_at_beginning(self):
data = int(input("Enter value to insert at beginning: "))
new_node = Node(data)
new_node.next = self.head
self.head = new_node
print(f"{data} inserted at the beginning.")
def insert_at_end(self, data=None):
if data is None:
data = int(input("Enter value to insert at end: "))
new_node = Node(data)
if self.head is None:
self.head = new_node
else:
temp = self.head
while temp.next:
temp = temp.next
temp.next = new_node
if data is not None:
print(f"{data} inserted at the end.")
def insert_at_index(self):
index = int(input("Enter index to insert at (0-based): "))
data = int(input("Enter value to insert: "))
if index == 0:
new_node = Node(data)
Ln: 167 Col: 0


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Deleted last node with value 26.
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 8
Number of nodes: 2
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 9
5 -> 10 -> None
----- Singly Linked List Menu -----
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traversal
10. Exit
Enter your choice: 10
Exiting program.
>>>
Ln: 155 Col: 0


lab-4.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/lab-4.py (3.13.7)
File Edit Format Run Options Window Help
class Node:
def __init__(self, data):
self.data = data
self.next = None
class SinglyLinkedList:
def __init__(self):
self.head = None
def create_list(self):
self.head = None
n = int(input("Enter number of nodes to create: "))
for i in range(n):
data = int(input(f"Enter value for node {i + 1}: "))
self.insert_at_end(data)
print("Linked list created successfully.")
def insert_at_beginning(self):
data = int(input("Enter value to insert at beginning: "))
new_node = Node(data)
new_node.next = self.head
self.head = new_node
print(f"{data} inserted at the beginning.")
def insert_at_end(self, data=None):
if data is None:
data = int(input("Enter value to insert at end: "))
new_node = Node(data)
if self.head is None:
self.head = new_node
else:
temp = self.head
while temp.next:
temp = temp.next
temp.next = new_node
if data is not None:
print(f"{data} inserted at the end.")
def insert_at_index(self):
index = int(input("Enter index to insert at (0-based): "))
data = int(input("Enter value to insert: "))
if index == 0:
new_node = Node(data)
Ln: 167 Col: 0

.

**EXPLANATION:-**

- The Node class stores a data value and a reference (next) to the following node in the list.
- The SinglyLinkedList class maintains a head pointer and provides methods for each menu operation.
- create_list() resets the list and repeatedly calls insert_at_end() to build it from user input.
- insert_at_beginning() creates a new node, points it to the current head, and makes it the new head.
- insert_at_end() traverses to the last node and links a new node after it, or sets it as head if the list is empty.
- insert_at_index() walks to the node just before the given position and relinks pointers to insert the new node there.
- delete_by_value() searches the list for a matching value and removes that node by updating the previous node's next pointer.
- delete_first_node() and delete_last_node() move the head pointer or unlink the final node after locating it by traversal.
- count_nodes() and display() both traverse the list from head to end, counting nodes or collecting their values.
- The main program runs an infinite while loop that prints the menu, reads the user's choice, and calls the matching method until option 10 (Exit) breaks the loop.

**ERRORS:-**

.

| SNO | Errors Observed | Rectification |
|---|---|---|
| 1 | Entering a non-integer value (e.g. letters) at any input prompt causes a ValueError. | Enter a valid integer whenever the program asks for a value, index, or choice. |
| 2 | Choosing an operation such as display, delete, or count before creating the list results in the 'List is empty.' message. | Select option 1 to create the list first before using the other operations. |
| 3 | Entering an index beyond the current size in 'Insert at specific index' prints 'Index out of range.' | Check the current node count (option 8) and enter an index within range. |
| 4 | Trying to delete a value that is not present in the list prints 'Value not found in list.' | Verify the value exists by displaying the list (option 9) before deleting it. |
| 5 | Entering a choice outside 1-10 shows 'Invalid choice. Try again.' | Re-enter a number between 1 and 10 from the menu. |

**TIME COMPLEXITY:-** Insert at beginning and delete first node are O(1); insert at end/index, delete by value/last node, count, and display are O(n)

**SPACE COMPLEXITY:-** O(n) for storing n nodes, O(1) auxiliary space per operation

**REASON:-** Operations that only touch the head (insert at beginning, delete first node) run in constant time because no traversal is needed, while operations that must reach a specific position, value, or the end of the list (insert at end/index, delete by value/last node, count, display) require walking through up to n nodes in the worst case.

.

# Week-5

1.Aim: Write a python program to implement a stack using an Array. Implement the stack operations such as push, pop, peak and also display the stack elements. PROGRAM: Stack using an Array

**Program:**

```
# Stack Implementation using Array


class Stack:
    def __init__(self):
        self.stack = []

    # Push operation
    def push(self, data):
        self.stack.append(data)
        print(data, "pushed into stack.")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow.")
        else:
            data = self.stack.pop()
            print(data, "popped from stack.")

    # Peek operation
    def peek(self):
```

```
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element:", self.stack[-1])

    # Display stack
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Stack elements:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])



# Main Program
s = Stack()

while True:
    print("\n========== STACK USING ARRAY ==========")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    print("=======================================")
```

.

```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter data: "))
    s.push(data)

elif choice == 2:
    s.pop()

elif choice == 3:
    s.peek()

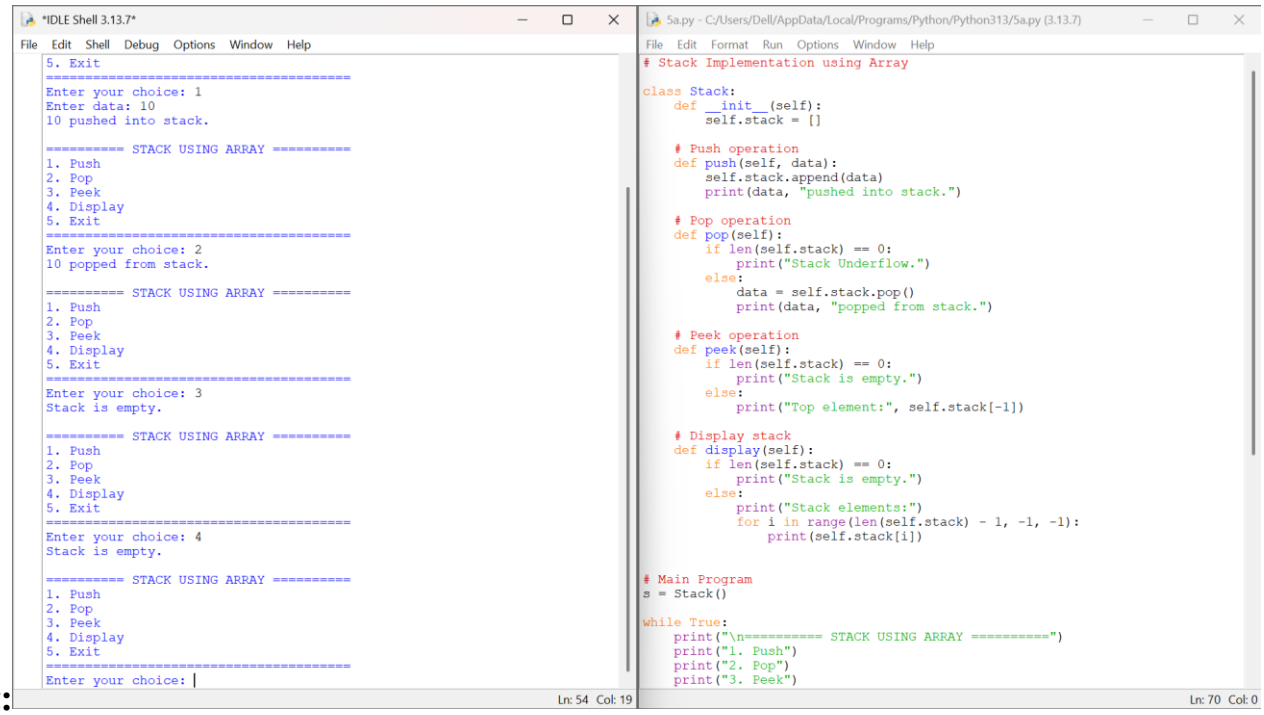
elif choice == 4:
    s.display()

elif choice == 5:
    print("Program terminated.")
    break

else:
    print("Invalid choice.")
```

.

**Output:**




**Explanation:**

1.stack = [] creates an empty list which is used as the array of the stack.

2.size stores the maximum number of elements the stack can hold, entered by the user.

3.The push(item) function first checks whether len(stack) == size. This is the overflow condition.

4.If the stack is full, the program prints "Stack Overflow" and the element is not inserted.

5.Otherwise, stack.append(item) inserts the element at the top of the stack.

6.The user enters the number of elements to push, and a for loop reads each element and calls push().

7.Finally, the stack is displayed from top to bottom using stack[::-1].

**Error table:**

| Si.no | Error obtained | Rectification |
|---|---|---|
| 1. | if len(stack) = size: | if len(stack) == size: |

.

2.Aim: Write a python program to implement a stack using an Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

Program: Stack using an Linked List.

```
# Stack Implementation using Linked List


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into stack.")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow.")
```

```
        else:
            data = self.top.data
            self.top = self.top.next
            print(data, "popped from stack.")

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty.")
        else:
            print("Top element:", self.top.data)

    # Display stack
    def display(self):
        if self.top is None:
            print("Stack is empty.")
        else:
            temp = self.top
            print("Stack elements:")

            while temp is not None:
                print(temp.data)
                temp = temp.next


# Main Program
```

```
s = Stack()

while True:
    print("\n======= STACK USING LINKED LIST =======")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    print("=======================================")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

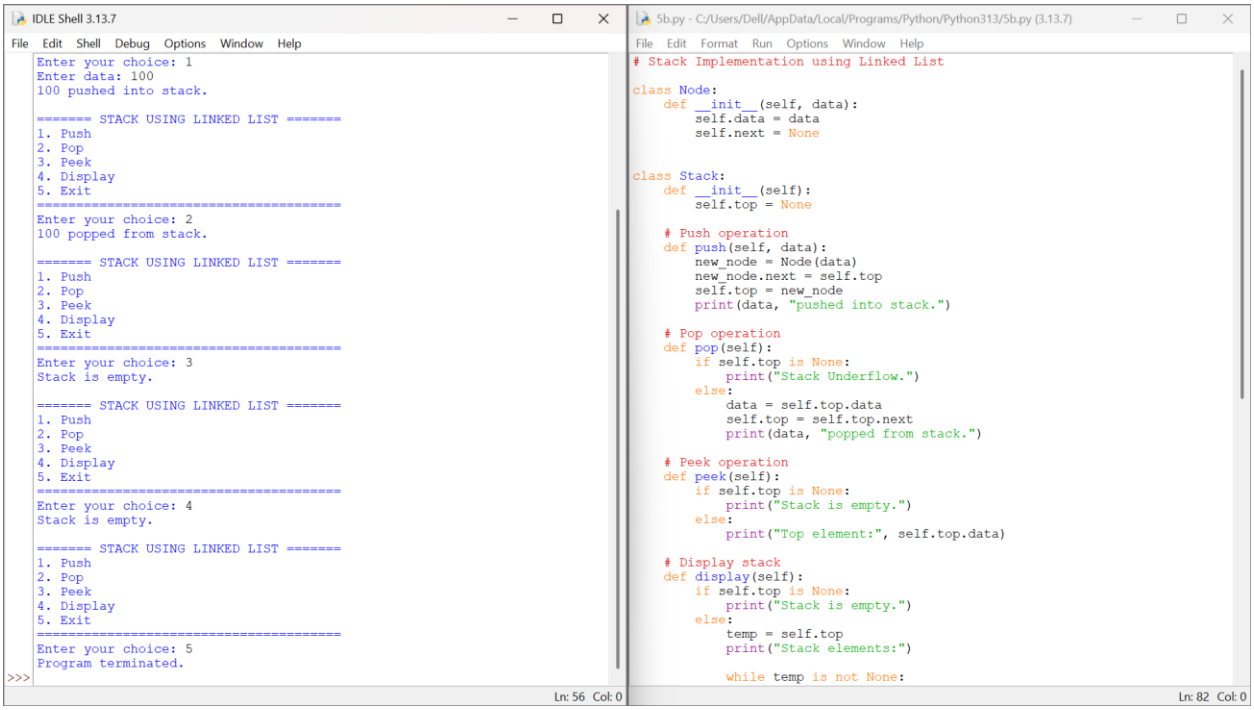
    elif choice == 4:
        s.display()
```

·

```
elif choice == 5:
    print("Program terminated.")
    break

else:
    print("Invalid choice.")
```

OUTPUT:




EXPLANATION:

Node(data) : Creates a node and stores the element in data. ·

next = None : Initially, the node has no link. ·

top = None : Initializes an empty stack.

Push:

·

· new_node = Node(data) : Creates a new node. ·

new_node.next = self.top : Links it to the current top. ·

self.top = new_node : Makes it the new top.

· Process: Create : Link : Update top.

Pop: ·

Checks self.top is None for underflow. ·

popped = self.top.data : Gets the top value. ·

self.top = self.top.next : Moves top to the next node. ·

Process: Get value : Move top : Remove old top.

Peek:

· self.top.data : Displays the top value without removing it.

Display:

· temp = self.top : Starts from top. ·

while temp : Traverses the nodes.

· temp = temp.next : Moves to the next node.

Main Program:

· SLL = Stack LinkedList() : Creates the stack. ·

choice : Selects Push, Pop, Peek, Display, or Exit. ·

while True : Repeats the menu. ·

break : Exits the program. ·

LIFO: Last inserted : First removed.

.

ERROR TABLE:

| s.no | Error Obtained | Rectification |
|---|---|---|
| 1. | Attribute Error may occur in Linked List Stack if self.top.data is accessed when top is None. | Check if self.top is None before performing pop() or peek(). |

Week-6

write a python program to implement 6A) a queue and 6B) a circular queue, each supporting enqueue, dequeue, peek and display through a menu-driven system, implemented using both an array and a linked list.

**6A) Queue**

1.Aim: Write a python program to implement a queue using an Array. Implement the queue operations such as enqueue, dequeue, peek and also display the queue elements, through a menu-driven system.

**Program:**

```
# Queue Implementation using Array


class Queue:
    def __init__(self):
        self.queue = []


    # Enqueue operation
    def enqueue(self, data):
        self.queue.append(data)
        print(data, "enqueued into queue.")
```

```
    # Dequeue operation
    def dequeue(self):
        if len(self.queue) == 0:
            print("Queue Underflow.")
        else:
            data = self.queue.pop(0)
            print(data, "dequeued from queue.")

    # Peek operation
    def peek(self):
        if len(self.queue) == 0:
            print("Queue is empty.")
        else:
            print("Front element:", self.queue[0])

    # Display queue
    def display(self):
        if len(self.queue) == 0:
            print("Queue is empty.")
        else:
            print("Queue elements:")
            for i in self.queue:
                print(i)

# Main Program
```

```
q = Queue()
while True:
    print("\n======= QUEUE USING ARRAY =======")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    print("=================================")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        q.enqueue(data)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```

•

## Output:

6A.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/6A.py (3.13.7)

File Edit Format Run Options Window Help

```
# Queue Implementation using Array

class Queue:
    def __init__(self):
        self.queue = []

    # Enqueue operation
    def enqueue(self, data):
        self.queue.append(data)
        print(data, "enqueued into queue.")

    # Dequeue operation
    def dequeue(self):
        if len(self.queue) == 0:
            print("Queue Underflow.")
        else:
            data = self.queue.pop(0)
            print(data, "dequeued from queue.")

    # Peek operation
    def peek(self):
        if len(self.queue) == 0:
            print("Queue is empty.")
        else:
            print("Front element:", self.queue[0])

    # Display queue
    def display(self):
        if len(self.queue) == 0:
            print("Queue is empty.")
        else:
            print("Queue elements:")
            for i in self.queue:
                print(i)

# Main Program
q = Queue()
while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Dell/AppData/Local/Programs/Python/Python313/6A.py =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data: 1
1 enqueued into queue.
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
1 dequeued from queue.
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data: 1
1 enqueued into queue.
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
1
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
>>>
```

## Explanation:

1.queue = [] creates an empty list which is used as the array of the queue.

.

2.enqueue(data) appends the element to the rear (end) of the list, so the oldest element always stays at index 0.

3.dequeue() checks whether the queue is empty; if not, queue.pop(0) removes and returns the front element, shifting the remaining elements left.

4.peek() displays queue[0], the front element, without removing it.

5.display() prints every element from front to rear using a for loop.

6.The menu-driven main program loops with while True, reading the user's choice and calling the matching method until option 5 (Exit) breaks the loop.

**Error table:**

| Si.no | Error obtained | Rectification |
|---|---|---|
| 1. | data = self.queue.pop() used to dequeue removed the last element (LIFO) instead of the front element. | data = self.queue.pop(0) removes the front element, giving correct FIFO order. |

2.Aim: Write a python program to implement a queue using a Linked List. Implement the queue operations such as enqueue, dequeue, peek and also display the queue elements, through a menu-driven system.

**Program:**

```
# Queue Implementation using Linked List


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
```

```
    self.front = None
    self.rear = None

# Enqueue operation
def enqueue(self, data):
    new_node = Node(data)
    if self.rear is None:
        self.front = new_node
        self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node
    print(data, "enqueued into queue.")

# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue Underflow.")
    else:
        data = self.front.data
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        print(data, "dequeued from queue.")

# Peek operation
```

```
    .

    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Front element:", self.front.data)

    # Display queue
    def display(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            temp = self.front
            print("Queue elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next

# Main Program
q = Queue()
while True:
    print("\n======= QUEUE USING LINKED LIST =======")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
print("=========================================")
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter data: "))
    q.enqueue(data)
elif choice == 2:
    q.dequeue()
elif choice == 3:
    q.peek()
elif choice == 4:
    q.display()
elif choice == 5:
    print("Program terminated.")
    break
else:
    print("Invalid choice.")
```

**Output:**

•

6B.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/6B.py (3.13.7)

File Edit Format Run Options Window Help

```
# Queue Implementation using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)
        if self.rear is None:
            self.front = new_node
            self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
        print(data, "enqueued into queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow.")
        else:
            data = self.front.data
            self.front = self.front.next
            if self.front is None:
                self.rear = None
            print(data, "dequeued from queue.")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Front element:", self.front.data)

    # Display queue
    def display(self):
        if self.front is None:
            print("Queue is empty.")
```

Ln: 77 Col: 0

6B.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/6B.py (3.13.7)

File Edit Format Run Options Window Help

```
            print(data, "dequeued from queue.")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Front element:", self.front.data)

    # Display queue
    def display(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            temp = self.front
            print("Queue elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next

# Main Program
q = Queue()
while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        q.enqueue(data)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```

Ln: 77 Col: 0

```
========== RESTART: C:/Users/Dell/AppData/Local/Programs/Python/Python313/6B.py ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data: 5
5 enqueued into queue.
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 5
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
5
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
>>>
```

Ln: 75 Col: 0

**Explanation:**

1.Node(data) creates a node holding a value and a next pointer; front and rear track the two ends of the queue.

2.enqueue(data) links a new node after rear and moves rear to it; if the queue was empty, front is also set to the new node.

3.dequeue() removes the node at front and moves front to front.next; if the queue becomes empty, rear is reset to None as well.

4.peek() displays front.data, the value at the front node, without removing it.

5.display() traverses the list from front to rear, printing each node's data.

6.The menu-driven main program loops with while True, reading the user's choice and calling the matching method until option 5 (Exit) breaks the loop.

**Error table:**

| Si.no | Error obtained | Rectification |
|---|---|---|
| 1. | After dequeuing the last node, rear still pointed to the removed node, causing the next enqueue to link to a stale node. | Add: if self.front is None: self.rear = None inside dequeue() to reset rear when the queue empties. |

.

**6B) Circular Queue**

3.Aim: Write a python program to implement a circular queue using an Array. Implement the queue operations such as enqueue, dequeue, peek and also display the queue elements, through a menu-driven system.

**Program:**

```
# Circular Queue Implementation using Array


class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1


    # Enqueue operation
    def enqueue(self, data):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow.")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = data
            print(data, "enqueued into queue.")
        else:
            self.rear = (self.rear + 1) % self.size
```

```
        self.queue[self.rear] = data
        print(data, "enqueued into queue.")

# Dequeue operation
def dequeue(self):
    if self.front == -1:
        print("Queue Underflow.")
    elif self.front == self.rear:
        data = self.queue[self.front]
        self.front = -1
        self.rear = -1
        print(data, "dequeued from queue.")
    else:
        data = self.queue[self.front]
        self.front = (self.front + 1) % self.size
        print(data, "dequeued from queue.")

# Peek operation
def peek(self):
    if self.front == -1:
        print("Queue is empty.")
    else:
        print("Front element:", self.queue[self.front])

# Display queue
def display(self):
```

```
        if self.front == -1:
            print("Queue is empty.")
        else:
            print("Queue elements:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size

# Main Program
size = int(input("Enter size of circular queue: "))
cq = CircularQueue(size)
while True:
    print("\n======= CIRCULAR QUEUE USING ARRAY =======")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    print("=============================================")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
```

```
        cq.enqueue(data)
    elif choice == 2:
        cq.dequeue()
    elif choice == 3:
        cq.peek()
    elif choice == 4:
        cq.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```

```
--- CIRCULAR QUEUE USING ARRAY ---
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
Queue is Empty.

--- CIRCULAR QUEUE USING ARRAY ---
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
Queue is Empty.

--- CIRCULAR QUEUE USING ARRAY ---
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
Queue is Elements.

--- CIRCULAR QUEUE USING ARRAY ---
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
Program terminated.
```

**Output:**

.

**Explanation:**

1.queue = [None] * size creates a fixed-size array of the given size, with front and rear initialised to -1 to mark an empty queue.

2.enqueue(data) checks (rear + 1) % size == front, the overflow condition; otherwise it advances rear circularly with the modulus operator and stores the data.

3.dequeue() checks front == -1 for underflow; otherwise it reads queue[front], advances front circularly, and resets both pointers to -1 if the last element was removed.

4.peek() displays queue[front], the front element, without removing it.

5.display() walks from front to rear using the modulus operator so it wraps correctly around the end of the array.

6.The circular (modulo-based) indexing lets the queue reuse the freed slots at the front of the array instead of leaving them unused, unlike a plain linear queue.

**Error table:**

| Si.no | Error obtained | Rectification |
|---|---|---|
| 1. | if self.rear + 1 == self.front: was used for the overflow check, so wraparound at the end of the array was never detected. | if (self.rear + 1) % self.size == self.front: correctly wraps the index before comparing. |

.Aim: Write a python program to implement a circular queue using a Linked List. Implement the queue operations such as enqueue, dequeue, peek and also display the queue elements, through a menu-driven system.

**Program:**

```
# Circular Queue Implementation using Linked List


class Node:
    def __init__(self, data):
```

```
        .
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)
        if self.front is None:
            self.front = new_node
            self.rear = new_node
            self.rear.next = self.front
        else:
            self.rear.next = new_node
            self.rear = new_node
            self.rear.next = self.front
        print(data, "enqueued into queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow.")
        elif self.front == self.rear:
```

```
        data = self.front.data
        self.front = None
        self.rear = None
        print(data, "dequeued from queue.")
    else:
        data = self.front.data
        self.front = self.front.next
        self.rear.next = self.front
        print(data, "dequeued from queue.")

# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display queue
def display(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        temp = self.front
        print("Queue elements:")
        while True:
            print(temp.data)
```

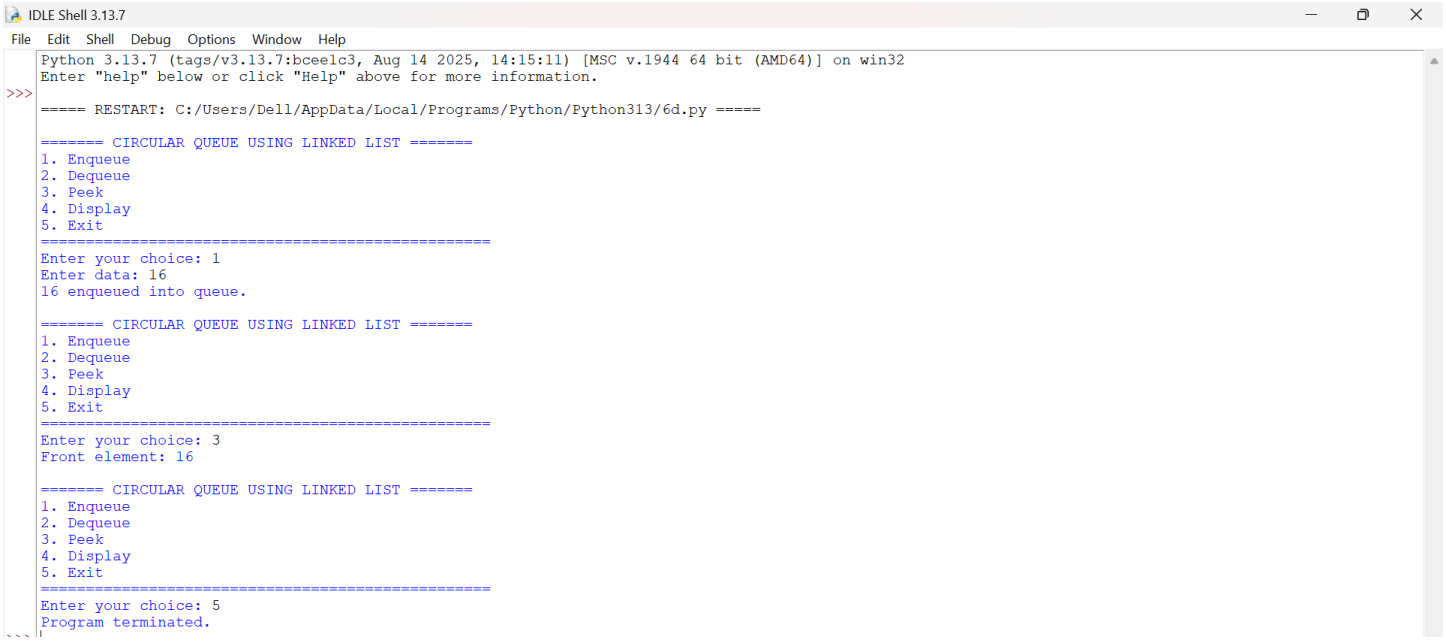
```
        temp = temp.next
        if temp == self.front:
            break

# Main Program
cq = CircularQueue()
while True:
    print("\n======= CIRCULAR QUEUE USING LINKED LIST =======")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    print("================================================")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        cq.enqueue(data)
    elif choice == 2:
        cq.dequeue()
    elif choice == 3:
        cq.peek()
    elif choice == 4:
        cq.display()
    elif choice == 5:
```

```
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```

**Output:**




**Explanation:**

1.Node(data) creates a node holding a value and a next pointer; front and rear track the two ends of the circular queue.

2.enqueue(data) links the new node after rear, moves rear to it, and always relinks rear.next back to front so the list stays circular.

3.dequeue() removes the node at front and moves front to front.next, relinking rear.next to the new front; if the queue empties, both pointers reset to None.

4.peek() displays front.data, the value at the front node, without removing it.

5.display() starts at front and traverses using next until it loops back to front, printing each node's data once.

.

6.Relinking rear.next to front after every insertion and deletion is what keeps the list circular; skipping it turns the structure back into a plain singly linked list.

**Error table:**

| Si.no | Error obtained | Rectification |
|---|---|---|
| 1. | rear.next = self.front was left out after inserting a new node, so the list stopped being circular. | Set self.rear.next = self.front again after every enqueue (and after every dequeue) to keep the last node linked back to the front. |

# WEEK – 7

**7A.1).AIM:** Write a Python Program to implement a binary tree using an array with all of its operations

## PROGRAM:

```
MAX = 100

tree = [None] * MAX

def insert(value, index=0): if index >= MAX: print("Tree is full.") return

if tree[index] is None:
    tree[index] = value
    print(value, "inserted.")
    return


if value < tree[index]:
    insert(value, 2 * index + 1)
else:
```

```
    .

    insert(value, 2 * index + 2)


def inorder(index=0): if index >= MAX or tree[index] is None: return

inorder(2 * index + 1)
print(tree[index], end=" ")
inorder(2 * index + 2)


def preorder(index=0): if index >= MAX or tree[index] is None: return

print(tree[index], end=" ")
preorder(2 * index + 1)
preorder(2 * index + 2)


def postorder(index=0): if index >= MAX or tree[index] is None: return

postorder(2 * index + 1)
postorder(2 * index + 2)
print(tree[index], end=" ")


def level_order(): for i in range(MAX): if tree[i] is not None: print(tree[i], end=" ")

def search(value, index=0): if index >= MAX or tree[index] is None: return False

if tree[index] == value:
    return True


if value < tree[index]:
    return search(value, 2 * index + 1)
else:
    return search(value, 2 * index + 2)


def count_nodes(index=0): if index >= MAX or tree[index] is None: return 0

return 1 + count_nodes(2 * index + 1) + count_nodes(2 * index + 2)
```

```
    .

def count_leaf_nodes(index=0): if index >= MAX or tree[index] is None: return 0

left = 2 * index + 1
right = 2 * index + 2

if (left >= MAX or tree[left] is None) and \
   (right >= MAX or tree[right] is None):
    return 1

return count_leaf_nodes(left) + count_leaf_nodes(right)


def height(index=0): if index >= MAX or tree[index] is None: return 0

left_height = height(2 * index + 1)
right_height = height(2 * index + 2)

return 1 + max(left_height, right_height)


def display():

        print ("\nArray Representation:")

        for i in range(MAX):

            if tree[i] is not None:

                print ("Index", i, ":", tree[i])

while True:

    print("\n===== BINARY TREE USING ARRAY =====")

    print("1. Insert")

    print("2. Inorder Traversal")

    print("3. Preorder Traversal")

    print("4. Postorder Traversal")
```

```
    .
    print("5. Level Order Traversal")
    print("6. Search")
    print("7. Count Nodes")
    print("8. Count Leaf Nodes")
    print("9. Height")
    print("10. Display Array")
    print ("11. Exit")
choice = int(input("Enter your choice: "))

if choice == 1:
   value = int(input("Enter value: "))
   insert(value)

elif choice == 2:
  print("Inorder:", end=" ")
   inorder()
   print()

elif choice == 3:
   print("Preorder:", end=" ")
   preorder()
   print()

elif choice == 4:
   print("Postorder:", end=" ")
   postorder()
   print()

elif choice == 5:
   print("Level Order:", end=" ")
   level_order()
```

```
    .

    print()

elif choice == 6:
    value = int(input("Enter value to search: "))
    if search(value):
        print("Value found.")
    else:
        print("Value not found.")

elif choice == 7:
    print("Total nodes:", count_nodes())

elif choice == 8:
    print("Leaf nodes:", count_leaf_nodes())

elif choice == 9:
    print("Height:", height())

elif choice == 10:
    display()

elif choice == 11:
    print("Program terminated.")
    break

else:
    print("Invalid choice.")
```

# OUTPUT:

7A.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/7A.py (3.13.7)

File Edit Format Run Options Window Help

```
MAX = 100
tree = [None] * MAX

def insert(value, index=0):
    if index >= MAX:
        print("Tree is full.")
        return

    if tree[index] is None:
        tree[index] = value
        print(value, "inserted.")
        return

    if value < tree[index]:
        insert(value, 2 * index + 1)
    else:
        insert(value, 2 * index + 2)


def inorder(index=0):
    if index >= MAX or tree[index] is None:
        return
    inorder(2 * index + 1)
    print(tree[index], end=" ")
    inorder(2 * index + 2)


def preorder(index=0):
    if index >= MAX or tree[index] is None:
        return
    print(tree[index], end=" ")
    preorder(2 * index + 1)
    preorder(2 * index + 2)


def postorder(index=0):
    if index >= MAX or tree[index] is None:
        return
    postorder(2 * index + 1)
    postorder(2 * index + 2)
    print(tree[index], end=" ")


def level_order():
    for i in range(MAX):
```

Ln: 15 Col: 0

```
    if tree[index] == value:
        return True

    if value < tree[index]:
        return search(value, 2 * index + 1)
    else:
        return search(value, 2 * index + 2)

def count_nodes(index=0):
    if index >= MAX or tree[index] is None:
        return 0
    return 1 + count_nodes(2 * index + 1) + count_nodes(2 * index + 2)


def count_leaf_nodes(index=0):
    if index >= MAX or tree[index] is None:
        return 0

    left = 2 * index + 1
    right = 2 * index + 2

    if ((left >= MAX or tree[left] is None) and
            (right >= MAX or tree[right] is None)):
        return 1

    return count_leaf_nodes(left) + count_leaf_nodes(right)


def height(index=0):
    if index >= MAX or tree[index] is None:
        return 0

    left_height = height(2 * index + 1)
    right_height = height(2 * index + 2)

    return 1 + max(left_height, right_height)


def display():
    print("\nArray Representation:")
    for i in range(MAX):
        if tree[i] is not None:
            print("Index", i, ":", tree[i])
```

Ln: 62 Col: 0

•

7A.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/7A.py (3.13.7)

File Edit Format Run Options Window Help

```
while True:
    print("\n===== BINARY TREE USING ARRAY =====")
    print("1. Insert")
    print("2. Inorder Traversal")
    print("3. Preorder Traversal")
    print("4. Postorder Traversal")
    print("5. Level Order Traversal")
    print("6. Search")
    print("7. Count Nodes")
    print("8. Count Leaf Nodes")
    print("9. Height")
    print("10. Display Array")
    print("11. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif choice == 2:
        print("Inorder:", end=" ")
        inorder()
        print()

    elif choice == 3:
        print("Preorder:", end=" ")
        preorder()
        print()

    elif choice == 4:
        print("Postorder:", end=" ")
        postorder()
        print()

    elif choice == 5:
        print("Level Order:", end=" ")
        level_order()
        print()

    elif choice == 6:
        value = int(input("Enter value to search: "))
        if search(value):
            print("Value found.")
```

Ln: 92 Col: 0

7A.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/7A.py (3.13.7)

File Edit Format Run Options Window Help

```
        print("Inorder:", end=" ")
        inorder()
        print()

    elif choice == 3:
        print("Preorder:", end=" ")
        preorder()
        print()

    elif choice == 4:
        print("Postorder:", end=" ")
        postorder()
        print()

    elif choice == 5:
        print("Level Order:", end=" ")
        level_order()
        print()

    elif choice == 6:
        value = int(input("Enter value to search: "))
        if search(value):
            print("Value found.")
        else:
            print("Value not found.")

    elif choice == 7:
        print("Total nodes:", count_nodes())

    elif choice == 8:
        print("Leaf nodes:", count_leaf_nodes())

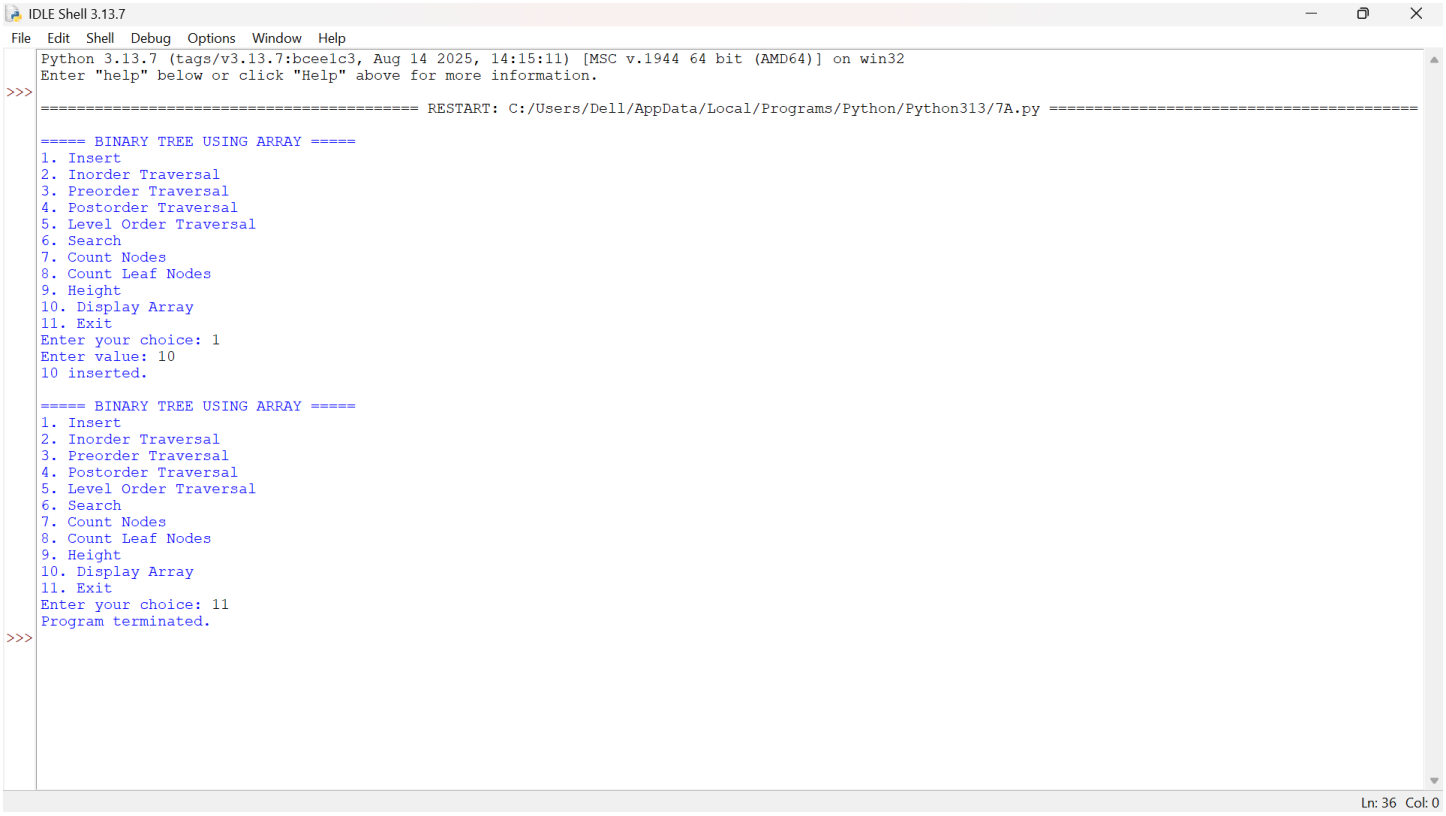
    elif choice == 9:
        print("Height:", height())

    elif choice == 10:
        display()

    elif choice == 11:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

Ln: 92 Col: 0




# Explanation:

1. **Array Representation:** The binary tree is stored in an array, where the root node is placed at index 0. For a node at index i, its left child is at 2i + 1 and the right child is at 2i + 2.
2. **Insertion:** The insert () function adds elements to the tree. In this program, smaller values are placed in the left subtree and larger values in the right subtree.
3. **Tree Traversals:** The program supports order, preorder, post order, and level-order traversal to visit all nodes in different orders.
4. **Searching and Counting:** The program can search for a particular value and calculate the total number of nodes and leaf nodes in the tree.
5. **Other Operations:** The program calculates the height of the tree and displays the array of representation. A menu-driven approach allows the user to select different operations.

# Error Table:

.

| S.No | Error | Correction |
|---|---|---|
| 1 | Using the wrong index for the left child. | Use 2 * index + 1 for the left child. |
| 2 | Using the wrong index for the right child. | Use 2 * index + 2 for the right child. |
| 3 | Accessing an array index beyond MAX. | Check if index >= MAX before accessing the array. |
| 4 | Not checking whether an array position is None. | Check if tree[index] is None before processing the node. |
| 5 | Forgetting to initialize the array. | Initialize it using tree = [None] * MAX. |

**7A.2).AIM:** Write a Python Program to implement a binary tree using an linked list with all of its operations

## PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self. Left = None
        self. Right = None
class BinaryTree:
```

```
    .

def __init__(self):
  self.root = None



def insert(self, value):
   new_node = Node(value)

   if self.root is None:
      self.root = new_node
      print(value, "inserted.")
      return

   queue = [self.root]

   while queue:
      current = queue.pop(0)

      if current.left is None:
         current.left = new_node
         print(value, "inserted.")
         return
      else:
         queue.append(current.left)

      if current.right is None:
         current.right = new_node
         print(value, "inserted.")
         return
      else:
         queue.append(current.right)



def inorder(self, node):
   if node is None:
      return
```

```
        .


    self.inorder(node.left)
    print(node.data, end=" ")
    self.inorder(node.right)


def preorder(self, node):
    if node is None:
        return

    print(node.data, end=" ")
    self.preorder(node.left)
    self.preorder(node.right)


def postorder(self, node):
    if node is None:
        return

    self.postorder(node.left)
    self.postorder(node.right)
    print(node.data, end=" ")


def level_order(self):
    if self.root is None:
        return

    queue = [self.root]

    while queue:
        current = queue.pop(0)

        print(current.data, end=" ")
```

```
        .

        if current.left is not None:
            queue.append(current.left)

        if current.right is not None:
            queue.append(current.right)


def search(self, node, value):
    if node is None:
        return False

    if node.data == value:
        return True

    return (self.search(node.left, value) or
            self.search(node.right, value))


def count_nodes(self, node):
    if node is None:
        return 0

    return (1 +
            self.count_nodes(node.left) +
            self.count_nodes(node.right))


def count_leaf_nodes(self, node):
    if node is None:
        return 0

    if node.left is None and node.right is None:
        return 1

    return (self.count_leaf_nodes(node.left) +
```

```
        .
            self.count_leaf_nodes(node.right))


def height(self, node):
    if node is None:
        return 0

    left_height = self.height(node.left)
    right_height = self.height(node.right)

    return 1 + max(left_height, right_height)


def minimum(self, node):
    if node is None:
        return None

    while node.left is not None:
        node = node.left

    return node.data


def maximum(self, node):
    if node is None:
        return None

    while node.right is not None:
        node = node.right

    return node.data

tree = BinaryTree()
```

```
.

while True:
    print("\n===== BINARY TREE USING LINKED LIST =====")
    print("1. Insert")
    print("2. Inorder Traversal")
    print("3. Preorder Traversal")
    print("4. Postorder Traversal")
    print("5. Level Order Traversal")
    print("6. Search")
    print("7. Count Nodes")
    print("8. Count Leaf Nodes")
    print("9. Height")
    print("10. Minimum")
    print("11. Maximum")
    print("12. Exit")
choice = int(input("Enter your choice: "))

if choice == 1:
  value = int(input("Enter value: "))
    tree.insert(value)

elif choice == 2:
    print("Inorder:", end=" ")
    tree.inorder(tree.root)
    print()

elif choice == 3:
    print("Preorder:", end=" ")
```

```
    .

    tree.preorder(tree.root)
    print()

elif choice == 4:
    print("Postorder:", end=" ")
    tree.postorder(tree.root)
    print()

elif choice == 5:
    print("Level Order:", end=" ")
    tree.level_order()
    print()

elif choice == 6:
    value = int(input("Enter value to search: "))

    if tree.search(tree.root, value):
        print("Value found.")
    else:
        print("Value not found.")

elif choice == 7:
    print("Total nodes:", tree.count_nodes(tree.root))

elif choice == 8:
    print("Leaf nodes:", tree.count_leaf_nodes(tree.root))

elif choice == 9:
    print("Height:", tree.height(tree.root))

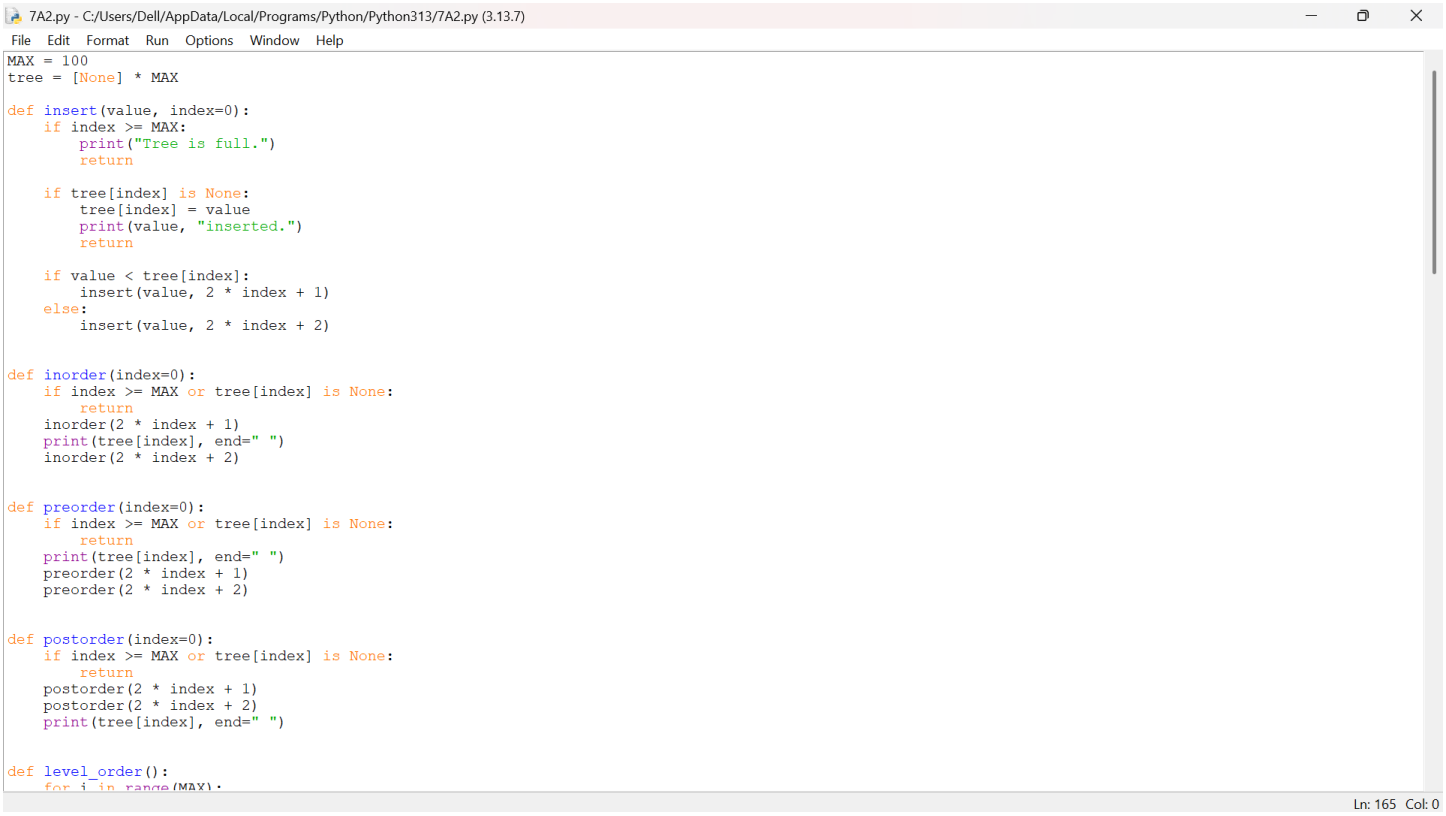
elif choice == 10:
    print("Minimum:", tree.minimum(tree.root))

elif choice == 11:
    print("Maximum:", tree.maximum(tree.root))
```

```
elif choice == 12:
    print("Program terminated.")
    break

else:
    print("Invalid choice.")
```

# OUTPUT:



- 

7A2.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/7A2.py (3.13.7)

File Edit Format Run Options Window Help

```
        if tree[i] is not None:
            print(tree[i], end=" ")


def search(value, index=0):
    if index >= MAX or tree[index] is None:
        return False

    if tree[index] == value:
        return True

    if value < tree[index]:
        return search(value, 2 * index + 1)
    else:
        return search(value, 2 * index + 2)


def count_nodes(index=0):
    if index >= MAX or tree[index] is None:
        return 0
    return 1 + count_nodes(2 * index + 1) + count_nodes(2 * index + 2)


def count_leaf_nodes(index=0):
    if index >= MAX or tree[index] is None:
        return 0

    left = 2 * index + 1
    right = 2 * index + 2

    if ((left >= MAX or tree[left] is None) and
            (right >= MAX or tree[right] is None)):
        return 1

    return count_leaf_nodes(left) + count_leaf_nodes(right)


def height(index=0):
    if index >= MAX or tree[index] is None:
        return 0

    left_height = height(2 * index + 1)
    right_height = height(2 * index + 2)

    return 1 + max(left_height, right_height)
```

Ln: 138 Col: 15

7A2.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/7A2.py (3.13.7)

File Edit Format Run Options Window Help

```
    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif choice == 2:
        print("Inorder:", end=" ")
        inorder()
        print()

    elif choice == 3:
        print("Preorder:", end=" ")
        preorder()
        print()

    elif choice == 4:
        print("Postorder:", end=" ")
        postorder()
        print()

    elif choice == 5:
        print("Level Order:", end=" ")
        level_order()
        print()

    elif choice == 6:
        value = int(input("Enter value to search: "))
        if search(value):
            print("Value found.")
        else:
            print("Value not found.")

    elif choice == 7:
        print("Total nodes:", count_nodes())

    elif choice == 8:
        print("Leaf nodes:", count_leaf_nodes())

    elif choice == 9:
        print("Height:", height())

    elif choice == 10:
        display()

    elif choice == 11:
        print("Program terminated.")
```

Ln: 138 Col: 15

•




# Explanation:

1. **Node Structure:** Each tree node is created using a Node class containing three parts: **data, left pointer, and right pointer**.
2. **Insertion:** The insert() function adds nodes using **level-order insertion**, filling the left and right positions of each level before moving to the next level.
3. **Tree Traversals:** The program provides **inorder, preorder, postorder, and level-order traversal** to display the elements of the binary tree.
4. **Searching and Counting:** The program can search for a value and calculate the **total number of nodes and leaf nodes** using recursive functions.
5. **Additional Operations:** The program calculates the **height** of the tree and finds the **minimum and maximum values**. A menu-driven system allows the user to perform each operation easily.

.

## Error Table:

| S.No | Error | Correction |
|---|---|---|
| 1 | Creating a node without initializing its child pointers. | Set self.left = None and self.right = None in the constructor. |
| 2 | Accessing node.data when node is None. | Add if node is None: return before accessing the node. |
| 3 | Forgetting to set the root when the tree is empty. | Use if self.root is None: self.root = new_node. |
| 4 | Not adding existing child nodes to the queue during insertion. | Add them using queue.append(current.left) and queue.append(current.right). |
| 5 | Calling traversal without passing the root node. | Call it as tree.inorder(tree.root), tree.preorder(tree.root), etc. |

# WEEK-8

**8A).AIM:** Write a Program to implement Binary Search Tree with it's operations Transversals also.

## Program:

```
class Node:
    def _init_(self, data):
        self.data = data
        self.left = None
```

```
        self.right = None

def insert(root, data): if root is None: return Node(data)

if data < root.data:
    root.left = insert(root.left, data)
elif data > root.data:
    root.right = insert(root.right, data)

return root


def search(root, key): if root is None or root.data == key: return root

if key < root.data:
    return search(root.left, key)

return search(root.right, key)


def find_min(root): while root.left: root = root.left return root

def delete(root, key): if root is None: return None

if key < root.data:
    root.left = delete(root.left, key)
elif key > root.data:
    root.right = delete(root.right, key)
else:
    if root.left is None:
        return root.right
    if root.right is None:
        return root.left

    temp = find_min(root.right)
    root.data = temp.data
    root.right = delete(root.right, temp.data)
```

```
    .
return root
 def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
root = None
values = [50, 30, 70, 20, 40, 60, 80]
for value in values:
    root = insert(root, value)
print("BST Inorder Traversal:")
inorder(root)
key = 40
if search(root, key):
    print("\n", key, "found in BST")
else:
    print("\n", key, "not found in BST")
root = delete(root, 30)
print("After deleting 30:")
inorder(root)
print()
```

•

# OUTPUT:

8A.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/8A.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)

    return root


def search(root, key):
    if root is None or root.data == key:
        return root

    if key < root.data:
        return search(root.left, key)

    return search(root.right, key)

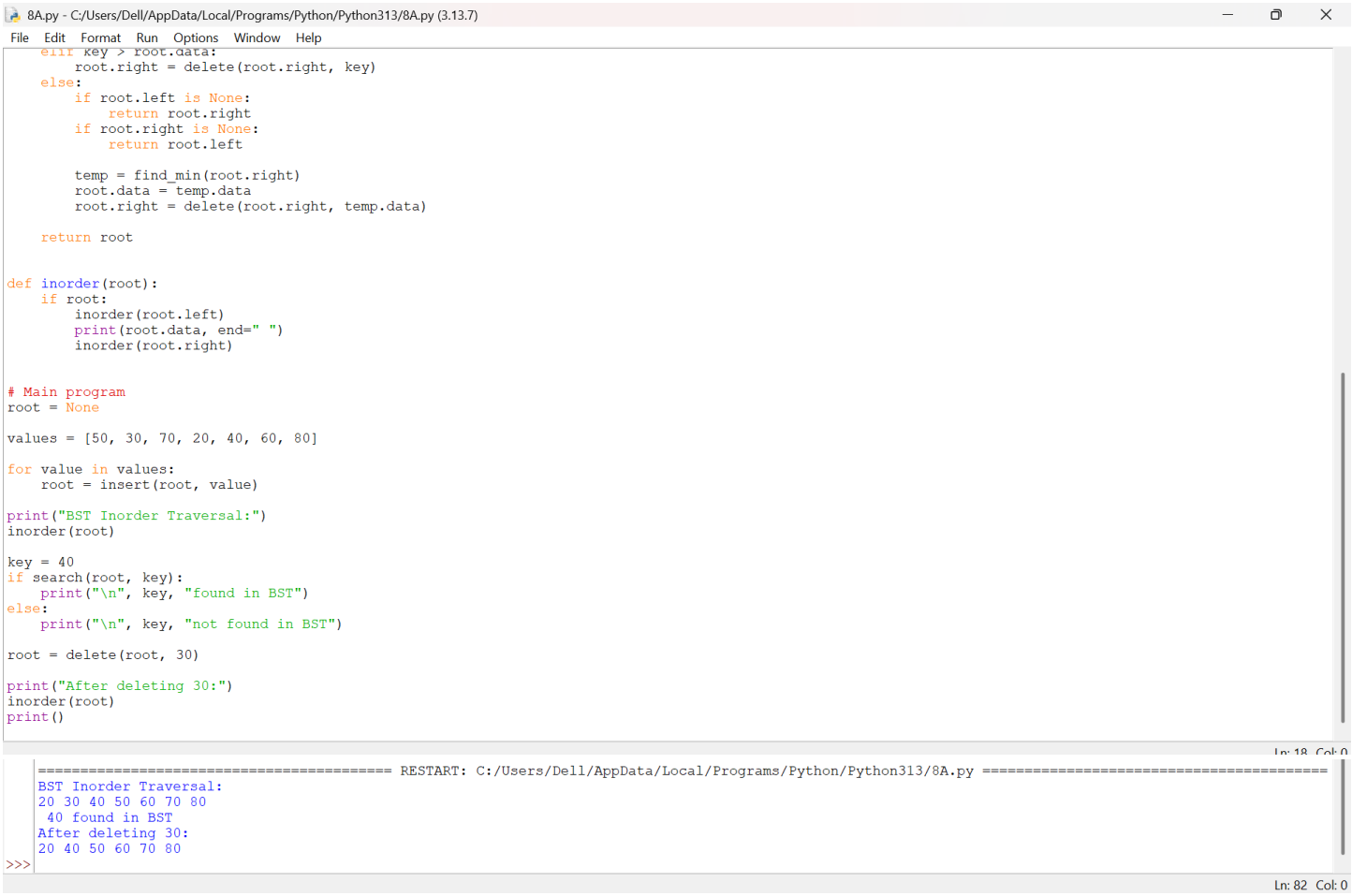

def find_min(root):
    while root.left:
        root = root.left
    return root


def delete(root, key):
    if root is None:
        return None

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
```

Ln: 8 Col: 11

•




## Explanation:

1. Node Creation: The Node class stores data and references to the left and right child nodes.

2. Insertion: The insert() function adds elements to the BST. Smaller values go to the left, and larger values go to the right.

3. Searching: The search() function checks whether a given value exists in the tree by comparing it with each node.

4. Deletion: The delete() function removes a node from the BST. It handles nodes with zero, one, or two children.

5. Inorder Traversal: The inorder() function visits the left subtree, root, and right subtree, displaying the elements in ascending order.

.

## Error Table:

| S.No | Error | Explanation |
|---|---|---|
| 1. | Not updating root | Always use root = insert(root, value) and root = delete(root, key) because these functions may return a new root. |
| 2. | Searching for a missing value | search() returns None when the key does not exist. Your if search(root, key) check handles this correctly. |
| 3. | Inserting duplicate values | insert() ignores duplicates because neither comparison is true when the values are equal. This is intentional behavior. |
| 4. | Deleting a node with two children | Simply removing the node could break the BST structure. Your code correctly uses find_min() to find its inorder successor. |
| 5. | Empty-tree minimum lookup | Calling find_min(None) directly would cause an error. Call it only when the subtree is non-empty, as your deletion code does. |

.

**8B).AIM:** Write a Program to implement AVL Tree with it's operations Transversals also.

## Program:

```
class Node:
        Def_init_(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1
def height(node):
      if node is None:
              return 0
      return node.height
def balance_factor(node):
       if node is None:
```

```
    .

            return 0

      return height(node.left) - height(node.right)

def right_rotate(y):

    x = y.left

    t2 = x.right

x.right = y
y.left = t2

y.height = 1 + max(height(y.left), height(y.right))
x.height = 1 + max(height(x.left), height(x.right))

return x


def left_rotate(x):

    y = x.right

    t2 = y.left

  y.left = x
  x.right = t2
x.height = 1 + max(height(x.left), height(x.right))
y.height = 1 + max(height(y.left), height(y.right))
return y
def insert(root, data):

    if root is None:

        return Node(data)
```

.

```
if data < root.data:
    root.left = insert(root.left, data)
elif data > root.data:
    root.right = insert(root.right, data)
else:
    return root
root.height = 1 + max(height(root.left), height(root.right))
bf = balance_factor(root)


if bf > 1 and data < root.left.data:
    return right_rotate(root)


if bf < -1 and data > root.right.data:
    return left_rotate(root)


if bf > 1 and data > root.left.data:
    root.left = left_rotate(root.left)
    return right_rotate(root)


if bf < -1 and data < root.right.data:
    root.right = right_rotate(root.right)
    return left_rotate(root)

return root
def search(root, key): if root is None or root.data == key: return root

if key < root.data:
    return search(root.left, key)
```

.

```
return search(root.right, key)


def find_min(root): while root.left: root = root.left return root

def delete(root, key): if root is None: return None

if key < root.data:
   root.left = delete(root.left, key)
elif key > root.data:
   root.right = delete(root.right, key)
else:
        if root.left is None:
              return root.right
   elif root.right is None:
      return root.left

   temp = find_min(root.right)
   root.data = temp.data
   root.right = delete(root.right, temp.data)

if root is None:
   return None

root.height = 1 + max(height(root.left), height(root.right))
bf = balance_factor(root)


if bf > 1 and balance_factor(root.left) >= 0:
   return right_rotate(root)
```

```
    .

if bf > 1 and balance_factor(root.left) < 0:
    root.left = left_rotate(root.left)
    return right_rotate(root)


if bf < -1 and balance_factor(root.right) <= 0:
    return left_rotate(root)


if bf < -1 and balance_factor(root.right) > 0:
    root.right = right_rotate(root.right)
    return left_rotate(root)

return root


def inorder(root):

    if root:

        inorder(root.left)

        print(root.data, end=" ")

        inorder(root.right)

root = None

values = [10, 20, 30, 40, 50, 25]

for value in values:

    root = insert(root, value)

print("AVL Inorder Traversal:")

inorder(root)
```

.

```
key = 25

if search(root, key):
    print("\n", key, "found in AVL tree")
else:
    print("\n", key, "not found in AVL tree")

root = delete(root, 40)

print("After deleting 40:")

inorder(root)

print()
```

**OUTPUT:**



- 

8B.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/8B.py (3.13.7)

File Edit Format Run Options Window Help

```
    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))

    return y


def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    else:
        return root

    root.height = 1 + max(height(root.left), height(root.right))
    bf = balance_factor(root)

    # LL case
    if bf > 1 and data < root.left.data:
        return right_rotate(root)

    # RR case
    if bf < -1 and data > root.right.data:
        return left_rotate(root)

    # LR case
    if bf > 1 and data > root.left.data:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    # RL case
    if bf < -1 and data < root.right.data:
        root.right = right_rotate(root.right)
        return left_rotate(root)

    return root


def search(root, key):
    if root is None or root.data == key:
        return root
```

Ln: 172 Col: 0

8B.py - C:/Users/Dell/AppData/Local/Programs/Python/Python313/8B.py (3.13.7)

File Edit Format Run Options Window Help

```
def find_min(root):
    while root.left:
        root = root.left
    return root


def delete(root, key):
    if root is None:
        return None

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = find_min(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    if root is None:
        return None

    root.height = 1 + max(height(root.left), height(root.right))
    bf = balance_factor(root)

    # LL case
    if bf > 1 and balance_factor(root.left) >= 0:
        return right_rotate(root)

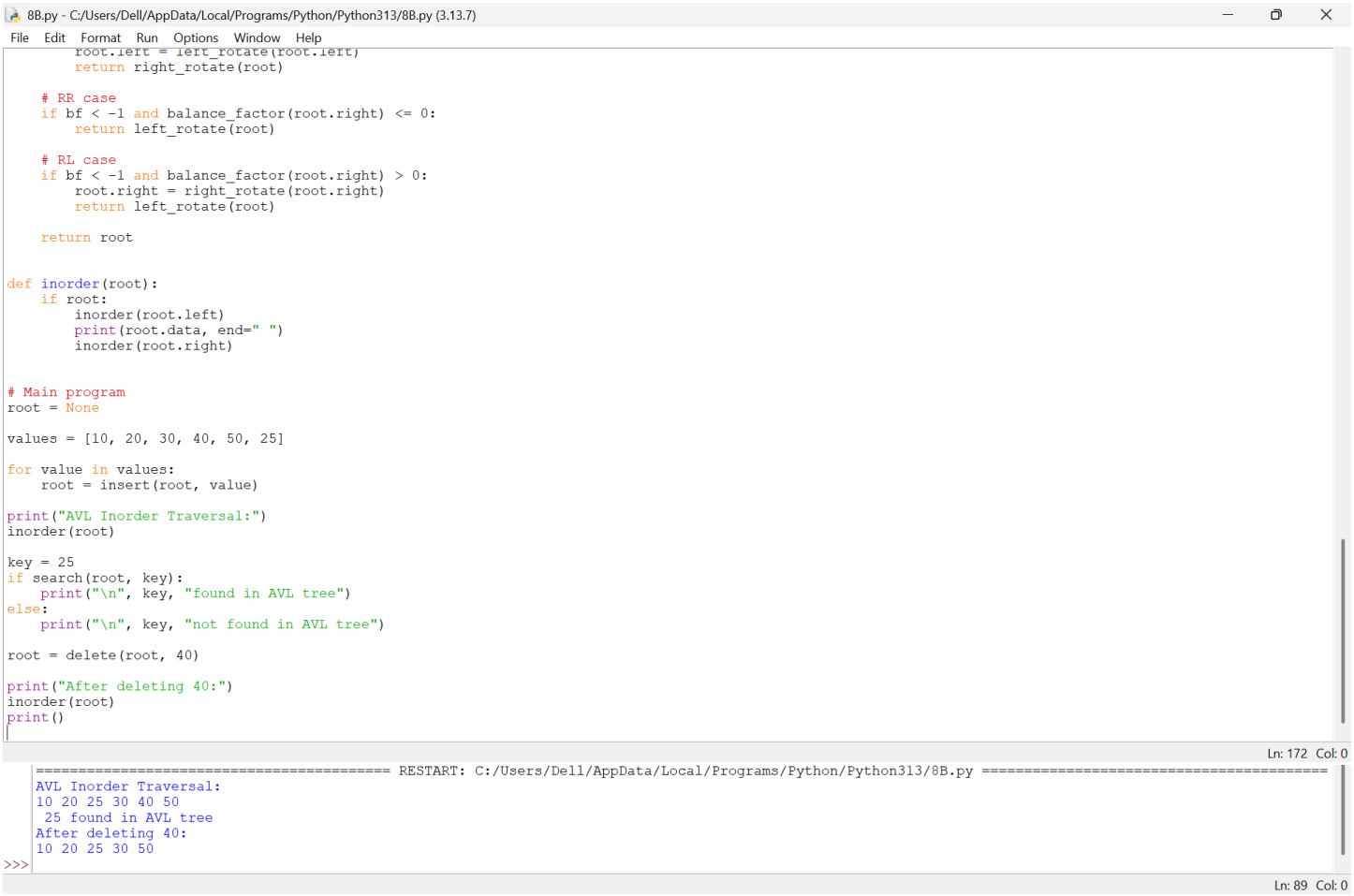
    # LR case
    if bf > 1 and balance_factor(root.left) < 0:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    # RR case
    if bf < -1 and balance_factor(root.right) <= 0:
        return left_rotate(root)

    # RL case
```

Ln: 172 Col: 0

•




## Explanation:

1. Node Creation and Height: The Node class stores data, left and right child references, and the height of each node.

2. Height and Balance Factor: The height() function calculates a node's height, while balance_factor() checks whether the tree is balanced.

3. Insertion and Rotations: The insert() function adds values to the AVL tree and performs LL, RR, LR, or RL rotations to maintain balance.

4. Searching and Deletion: The search() function finds a value, while delete() removes a node and rebalances the tree using rotations when necessary.

.

5. Inorder Traversal and Output: The inorder() function displays the elements in ascending order. The main program inserts values, searches for 25, deletes 40, and prints the updated tree.

## Error Table:

| S.No. | Error / Limitation | Explanation / Solution |
|---|---|---|
| 1. | Incorrect height update | If node heights are not updated after rotations, the balance factor becomes incorrect. Update heights after every rotation. |
| 2.. | Incorrect balance factor | Using height(right) - height(left) instead of height(left) - height(right) reverses the rotation conditions. |
| 3. | Missing rotation cases | Handling only LL and RR cases can leave the tree unbalanced. Handle all four cases: LL, RR, LR, and RL. |
| 4. | Root not updated | If the root changes after insertion, deletion, or rotation, assign the returned node to root. |
| 5. | Duplicate values | The insertion function ignores duplicate values using else: return root. Add separate duplicate-handling logic if duplicates are required. |

.

## WEEK-9

**9A).AIM:** Write a Program to implement AVL Tree with it's operations Transversals also.

**Program:**

```
class Node:
        Def_init_(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1
def height(node):
      if node is None:
            return 0
      return node.height
def balance_factor(node):
       if node is None:
            return 0
       return height(node.left) - height(node.right)
def right_rotate(y):
      x = y.left
      t2 = x.right
x.right = y
y.left = t2
```

```
    .

y.height = 1 + max(height(y.left), height(y.right))
x.height = 1 + max(height(x.left), height(x.right))

return x


def left_rotate(x):

    y = x.right

    t2 = y.left

  y.left = x
  x.right = t2
x.height = 1 + max(height(x.left), height(x.right))
y.height = 1 + max(height(y.left), height(y.right))
return y
def insert(root, data):

    if root is None:

        return Node(data)

if data < root.data:
   root.left = insert(root.left, data)
elif data > root.data:
   root.right = insert(root.right, data)
else:
   return root
root.height = 1 + max(height(root.left), height(root.right))
bf = balance_factor(root)


if bf > 1 and data < root.left.data:
   return right_rotate(root)
```

.

```
if bf < -1 and data > root.right.data:
    return left_rotate(root)


if bf > 1 and data > root.left.data:
    root.left = left_rotate(root.left)
    return right_rotate(root)


if bf < -1 and data < root.right.data:
    root.right = right_rotate(root.right)
    return left_rotate(root)

return root
def search(root, key): if root is None or root.data == key: return root
if key < root.data:
    return search(root.left, key)

return search(root.right, key)


def find_min(root): while root.left: root = root.left return root
def delete(root, key): if root is None: return None
if key < root.data:
    root.left = delete(root.left, key)
elif key > root.data:
    root.right = delete(root.right, key)
else:
    if root.left is None:
        return root.right
    elif root.right is None:
        return root.left
```

```
    .


    temp = find_min(root.right)
    root.data = temp.data
    root.right = delete(root.right, temp.data)

if root is None:
    return None

root.height = 1 + max(height(root.left), height(root.right))
bf = balance_factor(root)


if bf > 1 and balance_factor(root.left) >= 0:
    return right_rotate(root)


if bf > 1 and balance_factor(root.left) < 0:
    root.left = left_rotate(root.left)
    return right_rotate(root)


if bf < -1 and balance_factor(root.right) <= 0:
    return left_rotate(root)


if bf < -1 and balance_factor(root.right) > 0:
    root.right = right_rotate(root.right)
    return left_rotate(root)

return root


def inorder(root):
```

```
    .
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
root = None
values = [10, 20, 30, 40, 50, 25]
for value in values:
    root = insert(root, value)
print("AVL Inorder Traversal:")
inorder(root)
key = 25
if search(root, key):
    print("\n", key, "found in AVL tree")
else:
    print("\n", key, "not found in AVL tree")
root = delete(root, 40)
print("After deleting 40:")
inorder(root)
print()
```

**OUTPUT:**

```
AVL Inorder Traversal:
10 20 25 30 40 50
 25 found in AVL tree
After deleting 40:
10 20 25 30 50
```

.

**Explanation:**

1. Node Creation and Height: The Node class stores data, left and right child references, and the height of each node.

2. Height and Balance Factor: The height() function calculates a node's height, while balance_factor() checks whether the tree is balanced.

3. Insertion and Rotations: The insert() function adds values to the AVL tree and performs LL, RR, LR, or RL rotations to maintain balance.

4. Searching and Deletion: The search() function finds a value, while delete() removes a node and rebalances the tree using rotations when necessary.

5. Inorder Traversal and Output: The inorder() function displays the elements in ascending order. The main program inserts values, searches for 25, deletes 40, and prints the updated tree.

**Error Table:**

| S.No. | Error / Limitation | Explanation / Solution |
|---|---|---|
| 1. | Incorrect height update | If node heights are not updated after rotations, the balance factor becomes incorrect. Update heights after every rotation. |
| 2.. | Incorrect balance factor | Using height(right) - height(left) instead of height(left) - height(right) reverses the rotation conditions. |
| 3. | Missing rotation cases | Handling only LL and RR cases can leave the tree unbalanced. Handle all four cases: LL, RR, LR, and RL. |

.

| | | |
|---|---|---|
| 4. | Root not updated | If the root changes after insertion, deletion, or rotation, assign the returned node to root. |
| 5. | Duplicate values | The insertion function ignores duplicate values using else: return root. Add separate duplicate-handling logic if duplicates are required. |

## WEEK-10

**10A). AIM:** Write a python program to implement hashing using seperate chaining linear and Quardtic Probic.

.

**PROGRAM:**

```
size = 10
def chaining(keys):
    table = [[] for _ in range(size)]
    for key in keys:
    table [key % size]. append(key)
    print ("Separate Chaining:", table)

def linear(keys):
    table = [-1] * size
    for key in keys:
        i = key % size
        while table[i] != -1:
            i = (i + 1) % size
        table[i] = key
    print("Linear Probing:", table)

def quadratic(keys):
    table = [-1] * size
    for key in keys:
        h = key % size
        for i in range(size):
            j = (h + i * i) % size
```

```
        if table[j] == -1:
            table[j] = key
            break
        else:
            print("Table full for key:", key)
    print("Quadratic Probing:", table)

keys = list(map(int, input("Enter keys: ").split()))

chaining(keys)
linear(keys)
quadratic(keys)
```

**OUTPUT:**

```
Enter keys: 6
Separate Chaining: [[], [], [], [], [], [], [6], [], [], []]
Linear Probing: [-1, -1, -1, -1, -1, -1, 6, -1, -1, -1]
Quadratic Probing: [-1, -1, -1, -1, -1, -1, 6, -1, -1, -1]
```

**Explanation:**

1)Hash Function: key % size calculates the index where a key should be stored.

2)Separate Chaining: Uses a list at each index to store multiple keys when collisions occur.

3)Linear Probing: If a position is occupied, it checks the next position using (i + 1) % size.

4)Quadratic Probing: If a collision occurs, it checks positions using (h + i*i) % size.

.

5)Display: The program prints the hash table for all three methods so we can compare collision handling.

**Error Table:**

| S.No. | Error / Problem | Solution |
|---|---|---|
| 1. | IndexError: list index out of range | Use modulo % size to keep the index within the table. |
| 2. | ValueError: invalid literal | Enter only integer keys. |
| 3. | Infinite loop in linear probing | Check whether the hash table is full before inserting. |
| 4. | Key not inserted in quadratic probing | Check all possible probes and report if insertion fails. |
| 5. | Duplicate keys inserted | Allow duplicates or add a condition to reject them, depending on the requirement. |